Apunte_Maestro_Parcial1_TDI

Apunte Maestro — Parcial 1 de Teoría de la Información

*Refinado a partir de: apunte "PARCIAL-1 Apunte Completo", apuntes de cátedra (Fabricio Rubio), Área Temática 1 — Codificación de Fuente, la planilla de cátedra "Compresor PPM-C" y la **transcripción completa de los parciales anteriores** (2022 y 2023: enunciados Temas 1–4, resoluciones corregidas por el docente, recuperatorio, condicional y preguntas teóricas). Cubre unidades 1 a 5. Todos los cálculos numéricos fueron recalculados y verificados por computadora (fracciones exactas / simulación), no a mano, para eliminar errores de arrastre.*

*Actualización (oct-2026, 3.ª): se agregó **2.4.1** (cómo calcular $H(A)$, $H(B)$, $H(B|A)$, $H(A|B)$ con ejemplo y errores típicos) y las secciones **11.11 a 11.16** con el material de las carpetas «Parciales 1_Extras 2025» y «Recuperatorio o Condicionales 2025» (se descartó lo repetido).*

*Actualización (oct-2026, 2.ª): se agregó la **Sibling Property explicada a fondo** (6.7.1: definición, teorema que justifica su objetivo, numeración, regla de swap, en qué pasos se dispara, ejemplo `aeieo` del Tema 2) y el **Parcial del 07-10-2025** (Tema 1: Huffman, Markov, canal y discretización) resuelto y con la resolución de un alumno corregida (11.10).*

*Última actualización (oct-2026): se agregó el **recetario de métodos** (sección 10), los **ejercicios tipo resueltos paso a paso** — Markov de orden 1 con Huffman/Fano/Shannon, canal, ergodicidad, estacionaria, aritmética con 3 decimales, codificación predictiva (K óptimo), BWT inversa, cifrado, Huffman dinámico y su decodificación — (sección 11), y los errores que más marcó el docente (sección 12). Se corrigió el Huffman dinámico de ABRACADABRA: son **35 bits** (el resuelto a mano en el Excalidraw; antes decía 32), se corrigió la lectura de "3 decimales" en aritmética, la ergodicidad y el canal del Recuperatorio.*

Cómo usar este documento

El apunte tiene dos capas que conviene leer juntas:

1. **Teoría por unidad** (secciones 1 a 5): definiciones, fórmulas y el razonamiento de por qué cada fórmula es como es. Sirve para responder la parte teórica y para entender ejercicios de canal, muestreo y Markov.
2. **Ejemplo integrador único** (sección 6): la palabra `ABRACADABRA` se usa como hilo conductor y se comprime con **los métodos** que entran en el parcial — código de bloque, Huffman estático, Shannon, Fano, codificación aritmética, BWT, Huffman dinámico, PPM-C, Lempel-Ziv (ventana deslizante) y Lempel-Ziv-Welch — mostrando en cada uno el procedimiento, no solo el resultado. Al final hay una tabla comparativa que responde la pregunta "¿cuál comprimió mejor y por qué?".
3. **Recetario + ejercicios tipo** (secciones 10 a 12): la receta corta de cada método con sus trampas, y ejercicios completos con el formato exacto del parcial (por ejemplo *"codificar mediante Huffman, Fano y Shannon como fuente de Markov de orden 1"*, canal con información mutua, ergodicidad, aritmética con 3 decimales, K óptimo, BWT inversa, cifrado). **Si tenés poco tiempo: leé 0.1 (independiente vs. Markov), 10, 11 y 12.**

Se eligió `ABRACADABRA` porque es el mismo ejemplo que ya aparece en el material de cátedra para BWT y PPM-C, así que en vez de tener un ejemplo distinto por técnica (difícil de comparar), acá todos parten de la misma fuente y son directamente comparables entre sí.

---

0. Qué se evalúa en el Parcial 1 (mapa rápido)

El parcial se arma cruzando el examen modelo (4 variantes, temas 1–4), el parcial recuperatorio 2023, los apuntes de cátedra y el Área Temática 1. Estructura de las 4 variantes: siempre son **3 bloques** (fuente independiente, fuente de Markov, canal), cambiando el método de codificación exigido en el bloque A:

Variante	Método de codificación (bloque A)	Ejercicio de Markov	Canal BSC
Tema 1	Shannon	p(eeii)	C con p(0|0)=0,95
Tema 2	Huffman dinámico + Sibling Property	p(eiua)	C con p(0|0)=0,5
Tema 3	Aritmética (3 decimales)	p(uuaa)	I con p(0|0)=0,85, p(0)=0,7
Tema 4	LZ ventana deslizante (8 caract.)	p(aaii)	I con p(0|0)=0,85, p(1)=0,7

El Recuperatorio Tema 2 (2023) agrega dos elementos que no están en el modelo: un canal **BSC con** $p = 1/8$ del que se pide la información mutua $I(A;B) = H(B) - H(B|A)$ (con entrada equiprobable da $0{,}4564$ bits) y una **decodificación de Huffman dinámico** bit a bit (`000010010010011010` → `ANDABAN` , ver 11.8). Además tiene una fuente de Markov (`…EQUETENGUEEMEREQUEGUEGEJE…`) que se comprime con Shannon y con Fano, y dos teóricas (efecto de BWT previa y peor BSC).

Otros temas que aparecieron en los parciales de 2022-2023 y que NO están en la tabla de arriba (tenelos en el radar, están resueltos en la sección 11): discretización PCM/DPCM con "valor medio del segmento", **codificación predictiva con compuerta OREX y**

búsqueda del K óptimo, **encriptación** (transposición + Vigenère) con comparación de entropías, canal binario **asimétrico** (¿qué símbolo conviene enviar más?), canal ternario (mejor/peor/ruidoso), **BWT inversa** (`amnr$aaag`) y fuente con **base** $r \neq 2$ (entropía en base 5).

Parcial 07-10-2025 (Tema 1) — mismo esquema de 3 bloques: A) fuente independiente (redundancia + **Huffman** vs. código fijo), B) fuente de Markov de 1.er orden (entropía, ergodicidad, matriz de transición, **Huffman comparado con el independiente**), canal (**BSC con $p(0|0) = 0{,}9$ y peor canal ternario**) y una discretización **PCM 10 bits ±5 V + ADPCM con rango de 1 V** (7 bits). Resuelto en 11.10.

Material adicional 2025 incorporado: *Recuperatorio 28-10-2025 Tema 1* (Huffman dinámico `CANTABALA` , BWT de `AMARACHACATA$` con Huffman estático vs. Markov, K óptimo con fórmula, canal ternario: 11.11); *Parcial 07-10-2025 Tema 2* (BWT inversa `O$OOOOCLTMCRBO` , entropía de Markov de ambas cadenas, LZW, decodificación de Huffman dinámico `ROBODOS` , canal $0{,}8/0{,}15/0{,}05$: 11.12); *Temas 3 y 4* (trayectorias PPM-C `ABRA` / `ABA` , K óptimo, canal $0{,}9/0{,}05/0{,}05$, teórica $q = 6$: 11.13–11.14); *planillas de autocorrección* (11.15–11.16). **Cómo se calculan $H(A)$, $H(B)$, $H(B|A)$ y $H(A|B)$: 2.4.1.**

Lista cerrada de preguntas teóricas (tal como se repiten en los parciales corregidos):

1. Orden óptimo LZW + Huffman → primero LZW (saca redundancia de largo alcance), después Huffman (códigos cortos a lo frecuente).
2. Fuente de 7 símbolos con $p_i = r^{-l_i}$ para que Huffman=Shannon=Fano y Kraft=1, H=L.
3. Necesidad de la compansión → eficiencia de almacenamiento/transmisión y ahorro de costos.
4. Las 5 propiedades de I: positiva, inversa a p, crece con la cantidad de sucesos, aditiva, H máxima con equiprobabilidad.
5. PCM vs. DPCM: PCM cuantifica cada muestra; DPCM cuantifica la diferencia entre muestras (menor rango → menos bits, más robusto a ruido).
6. Peor canal uniforme ternario → todas las probabilidades en 1/3, $C = 0$; emisor y receptor se consideran independientes.
7. Métodos estadísticos estáticos: de doble lectura (cuentan y después codifican) y sin pérdida.
8. A mayor redundancia, mayor compactación posible.
9. Todo código instantáneo es unívoco; Huffman siempre produce un código instantáneo (prefijo).
10. Los árboles de Huffman estático y dinámico **no** tienen por qué coincidir.
11. Peor BSC: $p = 0{,}5 \Rightarrow C = 0$.
12. Los archivos comprimidos tienen mayor entropía y menor redundancia (se eliminó lo predecible).
13. BWT inversa: reconstruir la cadena original a partir de la transformada.
14. ¿Los árboles de Huffman estático y dinámico siempre coinciden? → **Falso** (sección 9, #5).
15. ¿Con BWT previa a la matriz de transición de Markov, cómo serían los resultados? → mejores (sección 9, #12).
16. ¿Los métodos estáticos son de doble lectura y con o sin pérdida? ¿Mayor redundancia → mayor compactación? ¿Instantáneo ⊂ unívoco? (Tema 3 y 4).
17. ¿Conviene comprimir antes o después de encriptar? → **primero comprimir, después encriptar** (sección 9, #13).
18. ¿Cuál es el mejor y el peor canal (binario y ternario)? (sección 9, #6 y #9, y 2.6).
19. Fuente de 4, 5, 6 o 7 símbolos con Huffman = Shannon = Fano → probabilidades $1/2, 1/4, 1/8, \ldots$ (sección 9, #2).

0.1 Cómo se redacta el parcial: fuente INDEPENDIENTE vs. fuente DEPENDIENTE (Markov)

Todos los enunciados de la cátedra traen **la misma secuencia dos veces**, en dos bloques, y la palabra clave del título del bloque decide **qué cuenta hay que hacer**:

El enunciado dice…	Qué significa	Qué se calcula
"Como fuente independiente" (bloque A / 1A)	**Memoria nula**: cada símbolo se trata solo, sin mirar el anterior. Se usa la **frecuencia de cada símbolo** $p_i = f_i/n$. Es el algoritmo "puro".	$H(S)$, H_{max}, redundancia, y **el algoritmo pedido** (Shannon, Huffman, Fano, Huffman dinámico, aritmética, LZ…) aplicado **directo sobre las** p_i (o sobre la cadena), con **un único código** para toda la fuente.
"Como fuente de Markov de primer orden / dependiente" (bloque B / 1B)	Cada símbolo depende del **anterior**. Se usan las **probabilidades condicionales** $P(s_j \mid s_i)$.	Matriz de transición, **entropía de Markov** $H(S_M)$, ¿ergódico?, matriz estacionaria π, probabilidad de una cadena ($p(eeii)$).
"Codificar … **como fuente de Markov de orden 1**" (con Huffman/Fano/Shannon)	Se junta lo anterior: primero se arma la matriz y después se aplica el algoritmo **una vez por estado**.	$L_M = \sum \pi_i L_i$ y la eficiencia con $H(S_M)$ (ver 5.3 y 11.2).

Regla práctica: si dice *independiente* ⇒ **no armes matriz de transición**. Si dice *Markov / dependiente* ⇒ **no uses las frecuencias globales para codificar**, usá las filas de la matriz.

Qué entra en cada bloque según el tema del examen modelo:

- **Bloque A (independiente)**: entropía → redundancia → *algoritmo del tema* (Tema 1 Shannon, Tema 2 Huffman dinámico, Tema 3 aritmética, Tema 4 LZ) → comparación con código de igual longitud.
- **Bloque B (Markov de 1.er orden)**: matriz de transición y entropía → ergódico → estacionaria → $p(\text{cadena})$. **En el examen modelo NO piden codificar** en este bloque; el algoritmo se aplica en el A.
- Otros parciales sí mezclan: el parcial 2022 pide "como fuente independiente: redundancia, Shannon…" y después "como fuente dependiente de primer orden: entropía, matriz de transición y estacionaria, ergódico". El Recuperatorio pide matriz de Markov (1.1) y después **comprimir con Shannon (1.2) y con Fano "considerando la fuente como independiente" (1.3)** y comparar (1.4): ahí el algoritmo va sobre las frecuencias globales aunque el ejercicio se llame "Markov". **Si el enunciado no aclara, leelo literal:** "considerando la fuente como independiente" ⇒ frecuencias; "como fuente de Markov" ⇒ por estado. Ante duda, resolvé las dos y rotulá cada una.

Las dos entropías — no se mezclan:

	Fuente independiente (memoria nula)	**Fuente dependiente (Markov orden 1)**
Nombre	$H(S)$, entropía de la fuente de memoria nula	$H(S_M) = H(B \mid A)$, entropía de Markov / condicional
Se calcula con	p_i = frecuencia de cada símbolo	cada **fila** de la matriz y la estacionaria π
Fórmula	$H(S) = \sum_i p_i \log_2 \frac{1}{p_i}$	$H(S_M) = \sum_i \pi_i \sum_j P(s_j \mid s_i) \log_2 \frac{1}{P(s_j \mid s_i)}$
Responde a	"¿cuánta incertidumbre hay en un símbolo **sin saber nada** del anterior?"	"¿cuánta incertidumbre queda en un símbolo **sabiendo el anterior**?"
Se usa para	redundancia $R = 1 - H/H_{max}$; eficiencia $\eta = H(S)/L$ de **un código único**	eficiencia $\eta = H(S_M)/L_M$ de **un código por estado**
Relación	$H(S_M) \leq H(S)$, con **igualdad solo si** el símbolo siguiente no depende del anterior (filas todas iguales a π)	

Con las vocales del Tema 1: $H(S) = 2{,}2516$ bits y $H(S_M) = 1{,}830$ bits. La diferencia ($0{,}42$ bits/símbolo) es **la información que aporta conocer el símbolo anterior**.

Cuál entropía va con cuál longitud (error típico):

- Código armado sobre frecuencias globales (Shannon/Huffman/Fano independiente) ⇒ L se compara con $H(S)$, y $H(S) \leq L < H(S) + 1$.
- Código por estado (Markov) ⇒ L_M se compara con $H(S_M)$, y $H(S_M) \leq L_M$. **No lo compares con** $H(S)$: L_M puede quedar *por debajo* de $H(S)$ (en el ejemplo $1{,}90 < 2{,}25$) y eso **no** viola el Teorema de Shannon, porque se está usando información extra (el estado).

Si piden Markov y *después* otro algoritmo — flujo completo:

1. Contar pares y armar P (dividir cada fila por las apariciones del origen).
2. π (estacionaria), ergodicidad.
3. $H(S_M)$ **(solo ahora, y solo porque es dependiente)**.
4. Aplicar el algoritmo pedido **a cada fila** con sus probabilidades condicionales > 0: Huffman/Fano/Shannon dan un código por estado; el primer símbolo del mensaje va con código de bloque.
5. $L_M = \sum \pi_i L_i$; $\eta = H(S_M)/L_M$; comparar con el código de bloque y con el mismo algoritmo en versión independiente (tabla de 11.2: bloque 3,0; Shannon indep. 2,667; Huffman indep. 2,333; Shannon Markov 2,467; Huffman/Fano Markov 1,900).

- Algoritmos de diccionario (LZ77/LZW) y aritmética/Huffman dinámico **no usan** la matriz: son siempre "sobre la cadena", se piden en el bloque independiente.

Markov de orden 2 (o m): mismo procedimiento, pero el **estado son los últimos m símbolos** (q^m estados; para vocales, 25 estados de orden 2) y cada fila da la distribución del siguiente símbolo. $H(S_M) = \sum_{\text{estados}} \pi_e \, H(\text{fila } e)$ y se codifica una tabla por estado. Cuanto mayor el orden, menor la entropía condicional (si hay dependencia real) pero más tablas y más datos para estimarlas. En el parcial el enunciado pide **orden 1**.

---

1. Unidad 1 — Información y entropía

1.1 Medida de información

$$I_i = \log_2\left(\frac{1}{p_i}\right) = -\log_2 p_i \quad [\text{bits}]$$

Un bit es la información de especificar 1 de 2 alternativas igualmente probables. Un suceso con $p = 1$ aporta $I = 0$ (no había incertidumbre que resolver).

Las 5 propiedades (pregunta teórica fija):

1. $I_i \geq 0$ siempre (se anula solo con $p_i = 1$).
2. Inversamente proporcional a p_i: lo raro informa mucho, lo seguro informa poco.
3. Crece con la cantidad de sucesos posibles.
4. Aditiva para sucesos independientes: $I(E_1 \cap E_2) = I(E_1) + I(E_2)$.
5. La entropía (información promedio) se maximiza con sucesos equiprobables.

1.2 Entropía (Shannon)

$$H(A) = \sum_{i=1}^{n} p_i I_i = -\sum_{i=1}^{n} p_i \log_2 p_i \quad [\text{bits/símbolo}]$$

Acotación: $0 \leq H(A) \leq \log_2 n$. El mínimo ($H = 0$) es una fuente determinística (un símbolo con $p = 1$); el máximo es equiprobabilidad total.

1.3 Redundancia y rendimiento

$$H_{max} = \log_2 q, \qquad \eta = \frac{H(A)}{H_{max}}, \qquad R = 1 - \eta$$

Interpretación: R mide qué fracción del "espacio" de símbolos se usa de más allá de lo estrictamente necesario para transportar la información que realmente hay.

Dos definiciones de redundancia que aparecen en los parciales — leé qué pide el enunciado:

- "Encuentre la **redundancia**" (fuente) → $R = 1 - \dfrac{H}{H_{max}} = 1 - \dfrac{H}{\log_2 q}$.
- "Compare el grado de compresión respecto de un código de **igual longitud** / longitud fija" → $\text{compresión} = 1 - \dfrac{L}{L_{fijo}}$, con $L_{fijo} = \lceil \log_2 q \rceil$ (y también se puede escribir $R = 1 - H/L_{fijo}$, que es lo que hizo un alumno en 2023 y se consideró válido porque estaba bien rotulado).

Entropía en base $r \neq 2$ (Parcial de Nanni Rebollo, alfabeto de 5 símbolos con $r = 5$): $H_r(S) = \sum p_i \log_r(1/p_i)$ se mide en **unidades de orden** r (no en bits). Para pasar de bits a base r: $H_r = H_2 / \log_2 r$. Con $r = 5$ y fuente `aeieeieouou aaaa` ($p_a = 5/15$, $p_e = 4/15$, $p_i = p_o = p_u = 2/15$): $H_5 = 0{,}9473$ y $R = 1 - H_5/1 = 5{,}26\%$ (el denominador es $\log_r q = \log_5 5 = 1$).

Cuenta típica del parcial (Tema 1, vocales, $n = 30$**,** $q = 5$**)** — fuente `a e i e o u a a i i u a o a u e e o o i i u u a o a a a a e` con frecuencias $a = 10,\ e = i = o = u = 5$:

$$H = \tfrac{1}{3}\log_2 3 + 4 \cdot \tfrac{1}{6}\log_2 6 = 0{,}5283 + 1{,}7233 = \mathbf{2{,}2516}\ \text{bits/sím.}, \quad H_{max} = \log_2 5 = 2{,}3219, \quad \eta = 96{,}97\%, \quad R = \mathbf{3{,}03\%}$$

(La planilla de cátedra redondea a $2{,}253$ y $R = 3\%$; es lo mismo con redondeo grueso.) Con respecto a un código de longitud fija de 3 bits: $R = 1 - 2{,}2516/3 = 24{,}9\%$.

(El cálculo completo de H, η, R sobre un ejemplo real se hace en la sección 6.1, con la propia palabra ABRACADABRA — mismo procedimiento que el ejemplo clásico de vocales del apunte de cátedra.)

---

2. Unidad 2 — Canal de información

2.1 El canal como matriz de transición

$$P(B|A) = \begin{bmatrix} P(b_1|a_1) & \cdots & P(b_s|a_1) \\ \vdots & \ddots & \vdots \\ P(b_1|a_q) & \cdots & P(b_s|a_q) \end{bmatrix}$$

Cada fila suma 1 (es una distribución de probabilidad condicionada a la entrada a_i).

2.2 Relaciones probabilísticas

$$P(b_j) = \sum_i P(a_i) P(b_j|a_i) \qquad \text{(prob. total)}$$

$$P(a_i|b_j) = \frac{P(a_i) P(b_j|a_i)}{P(b_j)} \qquad \text{(Bayes)}$$

2.3 Canal uniforme, simétrico, BSC

- **Uniforme**: todas las filas son la misma permutación.
- **Simétrico**: subconjunto del uniforme, con simetría también en columnas (el ruido afecta igual a cualquier símbolo).
- **BSC** (Canal Binario Simétrico): el caso particular más estudiado, matriz $\begin{bmatrix} 1-p & p \\ p & 1-p \end{bmatrix}$.

> ⚠️ En la transcripción de los parciales las resoluciones usan "simétrico" y "uniforme" a veces al revés (y un alumno escribió "simétrico, no uniforme" para un canal con filas $[0{,}95; 0{,}05]$ y $[0{,}35; 0{,}65]$). Para el examen: **la pregunta "¿es simétrico?" se responde mirando si cada fila es una permutación de las demás** (y las columnas también). El canal $\begin{bmatrix} 0{,}65 & 0{,}35 \\ 0{,}05 & 0{,}95 \end{bmatrix}$ **no** es simétrico (las filas no son permutación una de la otra).

Casos extremos (preguntas teóricas fijas):

Canal	**Matriz**	$H(B \mid A)$	$I(A;B)$	C
Mejor (sin ruido)	identidad (1 en la diagonal)	0	$H(A)$	$\log_2 q$ (binario: 1; ternario: 1,585)
Peor (BSC)	todo $0{,}5$	$= H(B)$	0	0
Peor (ternario uniforme)	todo $1/3$	$= H(B) = \log_2 3$	0	0
BSC con $p = 1$	$\begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$	0	$H(A)$	1 (invierte siempre: sigue siendo perfecto)

En el peor canal **emisor y receptor son independientes**: lo recibido no dice nada de lo enviado.

2.4 Entropías del sistema e información mutua

$$H(A,B) = H(A) + H(B|A) = H(B) + H(A|B)$$

$$I(A;B) = H(A) + H(B) - H(A,B) = H(A) - H(A|B) = H(B) - H(B|A)$$

- $H(A)$: incertidumbre a priori sobre la entrada.
- $H(B|A)$: **ruido** (lo que el canal agrega, información inútil).
- $H(A|B)$: **equivocación** (lo que sigue sin saberse del símbolo original después de leer la salida).
- $I(A;B) \geq 0$ siempre; $I = 0 \iff$ A y B independientes; es simétrica.

2.4.1 Cómo se calculan $H(A)$, $H(B)$, $H(B|A)$ y $H(A|B)$, paso a paso

Hay **cuatro entropías** y la confusión nace de que las cuatro usan la misma fórmula $\sum p \log_2 \frac{1}{p}$, pero **sobre distribuciones distintas** y con **ponderaciones distintas**. Esta tabla es el mapa:

Entropía	**¿Sobre qué distribución se calcula?**	**Fórmula**	**¿Se pondera?**
$H(A)$	las probabilidades de **entrada** $p(a_i)$ (las da el enunciado, o equiprobables)	$\sum_i p(a_i) \log_2 \frac{1}{p(a_i)}$	No
$H(B)$	las probabilidades de **salida** $p(b_j)$ (**hay que calcularlas**, no están en la matriz)	$\sum_j p(b_j) \log_2 \frac{1}{p(b_j)}$	No
$H(B \mid A)$ (**ruido**)	**cada fila** de la matriz por separado	$\sum_i p(a_i) \cdot \left[\sum_j p(b_j \mid a_i) \log_2 \frac{1}{p(b_j \mid a_i)} \right]$	**Sí: por** $p(a_i)$
$H(A \mid B)$ (**equivocación**)	cada **columna** tras aplicar Bayes	$\sum_j p(b_j) \cdot \left[\sum_i p(a_i \mid b_j) \log_2 \frac{1}{p(a_i \mid b_j)} \right]$	**Sí: por** $p(b_j)$

Procedimiento (siempre en este orden):

1. **Escribir la matriz** $P(B|A)$ (cada fila suma 1) y **fijar** $p(a_i)$: la que da el enunciado; si piden *capacidad de un canal uniforme/simétrico*, equiprobable $p(a_i) = 1/q$.
2. $H(A)$: aplicar la fórmula a $p(a_i)$. Equiprobable ⇒ $H(A) = \log_2 q$ (ternario $1{,}585$; binario 1).
3. **Probabilidades de salida** $p(b_j) = \sum_i p(a_i)\, p(b_j|a_i)$: es **el vector** $p(A)$ **multiplicado por la matriz**; cada b_j usa **la columna** j de la matriz, cada elemento multiplicado por la $p(a_i)$ de su fila. Comprobar que suman 1.
4. $H(B)$: la misma fórmula, pero con los $p(b_j)$ del paso 3. **No** con los números de la matriz.
5. $H(B|A)$ **(ruido)**: calcular la entropía de **cada fila** $H(\text{fila}_i) = \sum_j p(b_j|a_i) \log_2 \frac{1}{p(b_j|a_i)}$ y **promediar con** $p(a_i)$. Atajo: si las filas son permutaciones una de otra (canal uniforme/simétrico), todas valen lo mismo y $H(B|A) = H(\text{una fila})$, **sin dividir por** q (los q pesos $1/q$

ya suman 1).

6. **Información mutua**: $I(A;B) = H(B) - H(B|A)$. Verificación cruzada: $I = H(A) - H(A|B)$, y siempre $0 \le I \le \min(H(A), H(B))$.

Ejemplo completo con entrada NO equiprobable (para ver que $H(A) \neq H(B)$): canal ternario uniforme

$$P = \begin{bmatrix} 0{,}80 & 0{,}15 & 0{,}05 \\ 0{,}05 & 0{,}80 & 0{,}15 \\ 0{,}15 & 0{,}05 & 0{,}80 \end{bmatrix}, \qquad p(A) = (0{,}5;\ 0{,}3;\ 0{,}2)$$

- $H(A) = 0{,}5 \log_2 2 + 0{,}3 \log_2 \frac{1}{0{,}3} + 0{,}2 \log_2 \frac{1}{0{,}2} = 0{,}5 + 0{,}5211 + 0{,}4644 = \mathbf{1{,}4855}$
- **Salida**: $p(b_0) = 0{,}5 \cdot 0{,}80 + 0{,}3 \cdot 0{,}05 + 0{,}2 \cdot 0{,}15 = 0{,}445$; $p(b_1) = 0{,}5 \cdot 0{,}15 + 0{,}3 \cdot 0{,}80 + 0{,}2 \cdot 0{,}05 = 0{,}325$; $p(b_2) = 0{,}5 \cdot 0{,}05 + 0{,}3 \cdot 0{,}15 + 0{,}2 \cdot 0{,}80 = 0{,}230$ (suman 1 ✓).
- $H(B) = 0{,}445 \log_2 \frac{1}{0{,}445} + 0{,}325 \log_2 \frac{1}{0{,}325} + 0{,}230 \log_2 \frac{1}{0{,}230} = 0{,}5199 + 0{,}5271 + 0{,}4876 = \mathbf{1{,}5345}$
- **Ruido**: $H(\text{fila}) = 0{,}8 \log_2 \frac{1}{0{,}8} + 0{,}15 \log_2 \frac{1}{0{,}15} + 0{,}05 \log_2 \frac{1}{0{,}05} = 0{,}2575 + 0{,}4105 + 0{,}2161 = 0{,}8842$ y las 3 filas valen igual ⇒ $H(B|A) = (0{,}5 + 0{,}3 + 0{,}2) \cdot 0{,}8842 = \mathbf{0{,}8842}$.
- $I(A;B) = 1{,}5345 - 0{,}8842 = \mathbf{0{,}6503}$ **bits.** Verificación: Bayes da $H(A|B) = 0{,}8352$ y $H(A) - H(A|B) = 1{,}4855 - 0{,}8352 = 0{,}6503$ ✓.
- **Capacidad** (misma matriz, entrada equiprobable): $p(b_j) = 1/3 \Rightarrow H(B) = \log_2 3 = 1{,}5850 \Rightarrow C = 1{,}5850 - 0{,}8842 = \mathbf{0{,}7008}$ bits. (Atajo para canal uniforme: $C = \log_2 q - H(\text{fila})$.)

Errores concretos que aparecieron en las resoluciones de 2025 (revisá que no sean los tuyos):

Error	Qué hicieron	Por qué está mal	Correcto
Promediar una sola fila	Canal $0{,}8/0{,}1/0{,}1$: $H(B \mid A) = \frac{1}{3} \cdot 0{,}922 = 0{,}307 \Rightarrow C = 1{,}585 - 0{,}307 = 1{,}278$	$\frac{1}{3}$ es el peso de **una** fila; hay **tres** filas ($3 \cdot \frac{1}{3} = 1$)	$H(B \mid A) = 0{,}9219 \Rightarrow C = \mathbf{0{,}663}$
Misma cuenta + números cruzados	Canal $0{,}7/0{,}2/0{,}1$: usó $0{,}8 \cdot 0{,}322$ en lugar de $0{,}7 \cdot 0{,}515$ y dividió por 3 ⇒ $C = 1{,}2336$	escribió el $0{,}8$ del ejemplo anterior y repitió el $\frac{1}{3}$	$H(\text{fila}) = 1{,}1568 \Rightarrow C = \mathbf{0{,}4282}$
Señal de alarma	$C = 1{,}278$ con 80% de aciertos	un BSC con 90% de aciertos ya tiene $C = 0{,}53$; con ruido tan alto no se puede estar cerca del máximo $\log_2 3 = 1{,}585$	Comparar siempre con $\log_2 q$ y con un BSC parecido
$H(B)$ **con la matriz**	usar los $0{,}8; 0{,}1; 0{,}1$ como si fueran $p(b_j)$	$H(B)$ es la entropía de la **salida**, que sale de $p(A) \cdot P$	paso 3 y 4 de arriba
No pesar por $p(a_i)$ (Markov o canal)	sumar las entropías de fila sin ponderar	$H(B \mid A)$ y $H(S_M)$ son **promedios ponderados**	ver 11.12 (un alumno sumó las filas sin ponderar y dio $2{,}371$ en vez de $0{,}681$)
Canal "uniforme" mal armado	dar solo $p(0/0)$, $p(1/0)$ y poner $1/3$ en las otras filas	las otras filas de un canal **uniforme** son **permutaciones de la primera**, no $1/3$	filas $[0{,}8; 0{,}15; 0{,}05]$, $[0{,}05; 0{,}8; 0{,}15]$, $[0{,}15; 0{,}05; 0{,}8]$ ⇒ $C = 0{,}7008$
Capacidad de canal asimétrico con $1 - H(\bar{p})$	$\begin{bmatrix} 0{,}95 & 0{,}05 \\ 0{,}3 & 0{,}7 \end{bmatrix}$: promedió los errores $0{,}175 \Rightarrow C = 0{,}331$	esa fórmula es **solo para BSC**; hay que maximizar sobre $p(a)$	$C = \mathbf{0{,}3731}$ con $p(0) \approx 0{,}544$ (dar más probabilidad al 0)

Cómo se leen los datos según el verbo del enunciado: *"capacidad"* ⇒ canal uniforme/simétrico: entrada equiprobable, $C = \log_2 q - H(\text{fila})$; canal asimétrico: maximizar. *"información mutua"* con $p(a_i)$ dados ⇒ usar **esas** $p(a_i)$ (ejemplo de arriba). *"peor canal"* ⇒ todas las probabilidades $= 1/q$, $H(B|A) = H(B)$, $I = C = 0$.

Tabla de resultados de las capacidades que aparecieron (todas con entrada equiprobable salvo indicación):

Canal	$H(\text{fila})$	C
ternario uniforme $0{,}9/0{,}05/0{,}05$	$0{,}5690$	$\mathbf{1{,}0160}$
ternario uniforme $0{,}8/0{,}15/0{,}05$	$0{,}8842$	$\mathbf{0{,}7008}$
ternario uniforme $0{,}8/0{,}1/0{,}1$	$0{,}9219$	$\mathbf{0{,}6630}$
ternario uniforme $0{,}7/0{,}2/0{,}1$	$1{,}1568$	$\mathbf{0{,}4282}$
BSC $0{,}9/0{,}1$	$0{,}4690$	$\mathbf{0{,}5310}$
asimétrico $\begin{bmatrix} 0{,}9 & 0{,}1 \\ 0{,}3 & 0{,}7 \end{bmatrix}$	$0{,}4690$ y $0{,}8813$	$\mathbf{0{,}2967}$ con $p(0) \approx 0{,}528$ (conviene enviar más el **0**, que tiene menos ruido)

2.5 Capacidad de canal

$$C = \max_{P(a)} I(A;B) \qquad [\text{bits/símbolo}]$$

$$C_{BSC} = 1 - H(p) = 1 + p\log_2 p + (1-p)\log_2(1-p)$$

- $p = 0$ o $p = 1$: canal perfecto (con o sin inversión), $C = 1$.
- $p = 0{,}5$: canal inútil, $C = 0$ (nunca se sabe si un 1 recibido vino de un 0 o de un 1 enviado).

Canal asimétrico (no BSC): si las filas de la matriz no son todas la misma permutación, **no vale** $C = 1 - H(p)$. Hay que ir al método general:

1. Fijar $P(a_i)$.
2. Calcular $P(b_j) = \sum_i P(a_i)P(b_j|a_i)$.
3. Calcular $H(A)$, $H(B)$, y el ruido $H(B|A) = \sum_i P(a_i)H(\text{fila } i)$.
4. $I(A;B) = H(B) - H(B|A)$.
5. $C = \max_{P(a)} I(A;B)$ (no hay fórmula cerrada; se optimiza numéricamente, o se prueba con $P(a)$ equiprobable como primera aproximación).

Qué se pide según el verbo del enunciado (error recurrente de los alumnos):

- *"Encontrar la capacidad C"* → solo C. En un BSC es $1 - H(p)$ con entrada equiprobable; **no** mezcles datos como $P(0) = 0{,}7$ si no los dan (un alumno perdió puntos por eso).
- *"Encontrar la información mutua $I(A;B)$"* con $P(0)$ dado → usá **esa** $P(a)$, no la equiprobable.
- *"¿Qué símbolo conviene enviar con más probabilidad?"* (canal asimétrico, pregunta cualitativa) → el de la **fila con menos ruido** (menor $H(\text{fila})$): al usarlo más, baja $H(B|A)$ sin perder $H(B)$.

2.6 Ejemplos resueltos (verificados)

BSC con $p(0|0) = 0{,}95 \to p = 0{,}05$: $H(0{,}05) \approx 0{,}2864$, $C = 1 - 0{,}2864 = \mathbf{0{,}714}$ bits/símbolo.

BSC con $p(0|0) = 0{,}5 \to p = 0{,}5$: $C = 0$ (canal inútil).

BSC con $p(0|0) = 0{,}85$**,** $P(0) = 0{,}7$ (información mutua, no capacidad): $p = 0{,}15$. $P(0)_{sal} = 0{,}64$, $H(A) = 0{,}8813$, $H(B) = 0{,}9427$, $H(B|A) = 0{,}6098 \to I(A;B) = 0{,}8813 + 0{,}9427 - 1{,}4911 \approx \mathbf{0{,}333}$ bits.

BSC del Recuperatorio Tema 2 $\begin{bmatrix} 7/8 & 1/8 \\ 1/8 & 7/8 \end{bmatrix}$ — se pide $I(A;B) = H(B) - H(B|A)$. Entrada equiprobable: $P(b) = (1/2, 1/2)$, $H(B) = 1$, $H(B|A) = H(1/8) = \frac{7}{8}\log_2\frac{8}{7} + \frac{1}{8}\log_2 8 = 0{,}5436 \to I(A;B) = \mathbf{0{,}4564}$ bits (que además es C, por ser BSC).

> ⚠️ La resolución manuscrita de 2023 supuso $P(a_1) = 0{,}6,\ P(a_2) = 0{,}4$ (dato inventado) y llegó a $I(A;B) = -0{,}115$ bits. Es matemáticamente imposible: $I(A;B) \geq 0$ siempre. El docente lo marcó como incorrecto (anotó que correspondía $I(A;B) = H(A) - H(A|B)$ y que I nunca puede ser negativa). Si en un cálculo propio da negativo, hay un error de signo o de fórmula — revisar antes de entregar.
>
> *(Versión anterior de este apunte describía por error un canal "asimétrico" con $P(0|0) = 0$ para este recuperatorio; el enunciado real es el BSC de arriba. El método general para canales asimétricos sigue siendo válido y se usa en el ejemplo siguiente.)*

Canal asimétrico del parcial 10-10-2023 $\begin{bmatrix} 0{,}65 & 0{,}35 \\ 0{,}05 & 0{,}95 \end{bmatrix}$ (A: ¿es simétrico? B: ¿qué símbolo conviene enviar más?): **A)** No, las filas no son permutación una de otra. **B)** El **1**: su fila acierta el 95 % ($H(0{,}05) = 0{,}286$ bits de ruido) contra el 65 % de la fila del 0 ($H(0{,}35) = 0{,}934$). Numéricamente: con $P(0) = 0{,}5$ da $I = 0{,}3238$; el máximo está en $P(0) \approx 0{,}44$ ($P(1) \approx 0{,}56$) con $C \approx \mathbf{0{,}3265}$ bits. Se mira "cualitativamente", no hace falta calcular el máximo.

Canal ternario "ruidoso" $\begin{bmatrix} 0{,}8 & 0{,}1 & 0{,}1 \\ 0{,}1 & 0{,}8 & 0{,}1 \\ 0{,}1 & 0{,}1 & 0{,}8 \end{bmatrix}$ con entrada equiprobable: $H(B) = \log_2 3 = 1{,}585$, $H(B|A) = 0{,}8\log_2\frac{1}{0{,}8} + 2 \cdot 0{,}1\log_2\frac{1}{0{,}1} = 0{,}9219 \to C = I = \mathbf{0{,}663}$ bits. (Es simétrico, así que la equiprobable maximiza.)

> ⚠️ **Error clásico (Arce, 2023)**: pidieron el *peor* canal ternario y el alumno calculó $I = 1{,}0566$ con una $H(B|A)$ mal armada. Con todo en $1/3$: $H(B|A) = H(B) = \log_2 3$ y $I = 0$. Una capacidad mayor que 0 para el peor canal es una señal de que algo está mal.

Ejemplo completo de $I(A;B)$ con todas las entropías: sección 11.4.

---

3. Unidad 3 — Muestreo, cuantificación y codificación de la muestra

3.1 Muestreo (Nyquist)

$$f_s \geq 2B \quad (\text{frecuencia de Nyquist } f_N = 2B)$$

Si $f_s < 2B$ aparece **aliasing** (frecuencias falsas, irreconstruibles).

3.2 Cuantificación uniforme

$$\Delta = \frac{V_{max} - V_{min}}{L}, \qquad L = 2^n, \qquad -\frac{\Delta}{2} \le e_q \le \frac{\Delta}{2}$$

3.3 Tasa de bits

$$R = f_s \cdot n \cdot c$$

Caso	f_s	n	c	R
Telefonía	8 kHz	8	1	64 kbps
CD	44,1 kHz	16	2	≈1411 kbps

3.4 PCM vs. DPCM vs. ADPCM vs. modulación delta

- **PCM**: cuantifica y codifica cada muestra directamente.
- **DPCM**: codifica la *diferencia* entre muestras sucesivas. Como esa diferencia tiene rango dinámico menor, se necesitan menos bits para el mismo error. Se envía código PCM completo periódicamente para evitar que un error se propague indefinidamente.
- **ADPCM**: usa un predictor adaptativo (emisor y receptor comparten el mismo) — la diferencia predicción-muestra tiene rango aún menor.
- **Modulación delta**: DPCM de 1 bit (la señal solo sube o baja).

3.5 Discretización "con valor medio del segmento" — la trampa del parcial

Cuando el enunciado pide discretizar con un **error menor a** $\pm e$ y dice que el valor discreto es **el valor medio de cada segmento**:

$$\text{ancho del segmento} = 2e, \qquad \text{niveles} = \frac{\text{rango}}{2e}, \qquad n = \lceil \log_2(\text{niveles}) \rceil$$

(el error máximo es la **mitad** del ancho, $e_q = \Delta/2$). En **PCM** el rango es $V_{max} - V_{min}$ de la señal; en **DPCM** es el rango de las **diferencias entre muestras** (menor ⇒ menos bits para el mismo error).

Caso	**Rango**	e	**Ancho** $= 2e$	**Niveles**	**Bits**
DPCM ±1 V (Versión A, 10-10-2023)	2 V	0,005 V	0,01 V	$2/0{,}01 = 200$	$2^7 = 128 < 200 \le 256 = 2^8 \Rightarrow \mathbf{8}$
PCM ±5 V (Versión B, 10-10-2023)	10 V	0,005 V	0,01 V	$10/0{,}01 = 1000$	$2^9 = 512 < 1000 \le 1024 = 2^{10} \Rightarrow \mathbf{10}$

Verificación de la versión B: $10\ \text{V}/1024 = 0{,}009766$ V → error $= 0{,}00488 \le 0{,}005$ ✓. Y de la versión A con $n = 8$: $2/256 = 0{,}0078$ → error $0{,}0039 \le 0{,}005$ ✓.

> ⚠️ **Error que bajó la nota (50 %) a dos alumnas**: plantearon 400 niveles y 9 bits (usaron ancho $= e$ en vez de ancho $= 2e$). El docente anotó: *"el rango del segmento es ±0,005 = 0,01 → 2 V / 0,01 = 200 segmentos; con 256 llego: 2^8"*. La consigna "discutí con el grupo" apunta justamente a esto: **tomar el valor medio permite 1 bit menos** que sin esa condición.

Más casos (PCM de 16 bits, DPCM con rango $\pm 0{,}5$ V, error menor a $0{,}0001$ V) en 11.15.

3.6 Compansión (pregunta teórica fija)

Compansión = compresión + expansión. Antes de cuantificar se **comprime** la dinámica de la señal con una curva no lineal (en telefonía, ley μ o ley A) y el receptor la **expande** con la curva inversa. El efecto neto es una **cuantificación no uniforme**: más resolución (segmentos más finos) para las amplitudes **pequeñas**, que son las más frecuentes (voz), y menos para las grandes. **Necesidad**: mantener la relación señal/ruido de cuantificación pareja para señales débiles y fuertes con pocos bits por muestra ⇒ eficiencia de almacenamiento/transmisión y ahorro de costos.

3.7 Ejemplo resuelto del examen condicional (PCM vs. DPCM, 5 minutos de audio)

Señal $[-10, 10]$ V, $\Delta = 2\,\mu V$. Error de cuantización $e_q = \Delta/2 = 1\,\mu V$. PCM: $L = 20\text{V}/2\mu V = 10^7$ niveles → $n = \lceil \log_2 10^7 \rceil = \mathbf{24}$ bits/muestra. DPCM (rango entre muestras 2V, mismo Δ): $L = 10^6 \rightarrow n = \mathbf{20}$ bits/muestra. Con $f_{max} = 2$kHz ($f_s = 4$kHz) y 5 min de audio: PCM = 96 kbps ($2{,}88 \times 10^7$ bits); DPCM = 80 kbps ($2{,}4 \times 10^7$ bits) → **DPCM ahorra ≈17%** manteniendo el mismo error.

---

4. Unidad 4 — Códigos de fuente: teoría

4.1 Clasificación de códigos

- **Bloque** (longitud fija, ej. ASCII) vs. **no bloque** (longitud variable, corto para lo frecuente).
- **Singular** (dos símbolos comparten palabra código → inútil) vs. **no singular**.
- **Unívoco**: cada secuencia codificada tiene una única descomposición posible.
- **Instantáneo / prefijo**: subconjunto de los unívocos; se puede decodificar apenas se completa cada palabra, porque ninguna es prefijo de otra. Todo instantáneo es unívoco (no al revés).

4.2 Desigualdad de Kraft

$$\sum_{i=1}^{q} r^{-l_i} \leq 1$$

- $= 1$: instantáneo, árbol de prefijos lleno, y ahí $H = L$ (código compacto).
- < 1: instantáneo, sobra espacio en el árbol.
- > 1: no existe código instantáneo con esas longitudes.

McMillan es la misma desigualdad pero como condición *necesaria* (no suficiente) para códigos unívocos — puede cumplirse Kraft con un código que no sea instantáneo pero sí unívoco.

4.3 Longitud promedio, eficiencia, Primer Teorema de Shannon

$$L = \sum_i p_i l_i, \qquad \eta = \frac{H(X)}{L}, \qquad R = 1 - \eta$$

$$H(X) \leq L < H(X) + 1$$

Un código es **compacto** (óptimo, $\eta = 100\%$) cuando $p_i = r^{-l_i}$ para todo i (longitudes enteras exactas).

4.4 Los tres métodos estáticos — el procedimiento, en abstracto

Huffman (óptimo — minimiza L):

1. Ordenar probabilidades de mayor a menor.
2. Combinar los r símbolos menos probables en un nodo cuya probabilidad es la suma.
3. Reordenar y repetir hasta que quede un único nodo (raíz, prob=1).
4. Asignar 0/1 a cada rama y leer el código de raíz a hoja.

Shannon: $l_i = \lceil \log_2(1/p_i) \rceil$. Se calcula la función acumulada (FA) de cada símbolo, se pasa a binario, y se toman los primeros l_i dígitos.

- La condición del enunciado es $\log_2 \frac{1}{p_i} \leq l_i < \log_2 \frac{1}{p_i} + 1$: l_i es el **primer entero** $\geq \log_2(1/p_i)$.
- ⚠️ **Si $\log_2(1/p_i)$ ya es un entero, l_i es ese entero** (no se suma 1). Usar la "cota superior" ahí fue el error marcado en Arce: para $p = 4/16$, $\log_2 4 = 2 \Rightarrow l = 2$ (no 3), y con eso $L_{Shannon} = 2{,}75 = L_{Huffman}$.
- **FA**: es la suma de las probabilidades de los símbolos **anteriores** (el primero vale 0), con los símbolos ordenados de mayor a menor probabilidad.
- **FA → binario** por multiplicaciones sucesivas por 2: se multiplica la parte decimal por 2; la parte entera del resultado es el siguiente dígito binario. Ej.: $0{,}3077 \to 0{,}6154\ (0) \to 1{,}2308\ (1) \to 0{,}4616\ (0) \to 0{,}9232\ (0) \Rightarrow 0{,}0100\ldots$; con $l = 3$ queda `010`.
- La longitud promedio de Shannon puede **pasarse de la de Huffman**; el código sigue siendo instantáneo.

Fano: ordenar por probabilidad decreciente, dividir recursivamente en dos grupos cuya suma de probabilidades sea lo más parecida posible a 0,5 cada uno, asignando 0 a un grupo y 1 al otro, y repitiendo dentro de cada grupo.

- Si hay **dos cortes igual de buenos**, cualquiera es válido (en `ABRACADABRA` el propio Excalidraw anota "2 formas de dividir"); la longitud promedio puede cambiar según el corte, probá ambos y quedate con el menor L si te dejan elegir.
- ⚠️ **Fano ≠ Huffman**: en el Recuperatorio un alumno comprimió "con Fano" usando Huffman y el docente anotó "NO. Comprimió x Huffman". Leé qué algoritmo piden.

Armar un código a partir de longitudes / verificar que es instantáneo: (1) calcular $\sum 2^{-l_i}$ (Kraft): si es ≤ 1 existe un código instantáneo con esas longitudes; (2) para comprobar un código dado, mirar que **ninguna palabra sea prefijo de otra** (equivale a que todas sean hojas del árbol). Huffman, Shannon y Fano siempre producen códigos instantáneos.

Cuándo coinciden los tres métodos (y $H = L$): cuando todas las $p_i = 2^{-l_i}$. Ejemplos de distribuciones válidas (todas con $\sum 2^{-l_i} = 1$): 4 símbolos $\frac{1}{2}, \frac{1}{4}, \frac{1}{8}, \frac{1}{8}$; 5 símbolos $\frac{1}{2}, \frac{1}{4}, \frac{1}{8}, \frac{1}{16}, \frac{1}{16}$; 6 símbolos $\frac{1}{2}, \frac{1}{4}, \frac{1}{8}, \frac{1}{16}, \frac{1}{32}, \frac{1}{32}$; 7 símbolos $\frac{1}{2}, \frac{1}{4}, \frac{1}{8}, \frac{1}{16}, \frac{1}{32}, \frac{1}{64}, \frac{1}{64}$. La regla: **las dos últimas probabilidades son iguales** y todas son potencias de 2 que suman 1. (Cuidado: ⚠️ una alumna propuso $\frac{1}{4}, \frac{1}{8}, \frac{1}{16}, \frac{1}{32}, \frac{1}{32}$ para 5 símbolos y suma $15/32 \neq 1$: "debe sumar 1".)

Relación entre los tres:

$$L_{Huffman} \leq L_{Fano} \leq L_{Shannon}$$

Huffman nunca es peor que los otros dos; la igualdad ocurre cuando $p_i = r^{-l_i}$ exactamente (fuente "compatible" con potencias de 2).

4.5 Extensión de orden n

Agrupar símbolos en bloques de n reduce el desperdicio por redondeo (que en el peor caso es 1 bit *por símbolo codificado*, no por bloque):

$$H(S^n) = n \cdot H(S), \qquad H(S) \leq \frac{L_n}{n} < H(S) + \frac{1}{n}, \qquad \lim_{n\to\infty} \frac{L_n}{n} = H(S)$$

---

5. Unidad 5 — Fuentes dependientes y códigos avanzados: teoría

5.1 Fuente de Markov

Orden m: $P(X|X_{m-1}, \ldots, X_0)$. Estados = últimas m emisiones, hay q^m estados posibles. Matriz de transición: cada fila suma 1, $\sum_j P(S_j|S_i) = 1$.

5.2 Distribución estacionaria y ergodicidad

Un proceso es **ergódico** si, a largo plazo, el promedio temporal (una trayectoria larga) coincide con el promedio de conjunto (muchas realizaciones). En una cadena finita eso equivale a: **desde cualquier estado se puede llegar a cualquier otro** (irreducible) **y** no queda atrapada en un ciclo periódico (aperiódica). En la práctica: *dibujá el grafo de transiciones (flecha $i \to j$ si $P_{ij} > 0$) y verificá que se llega de todo estado a todo estado*; o comprobá que alguna potencia P^k tiene **todos** los elementos positivos (matriz **regular**). Una cadena ergódica tiene una **única** distribución estacionaria, y esa distribución es la frecuencia de largo plazo de cada estado.

La distribución estacionaria π cumple:

$$\pi P = \pi, \qquad \sum_i \pi_i = 1$$

Tres formas de hallarla: (1) iterativa — elevar P a una potencia grande hasta que converja; (2) analítica — resolver el sistema de ecuaciones + normalizar; (3) álgebra lineal — autovector de autovalor 1, normalizado. En la **matriz estacionaria** cada una de las q filas es el vector π repetido.

> ⚠️ **Que la matriz tenga ceros NO alcanza para decir "no es ergódico".** En las resoluciones de 2023 los alumnos escribieron "no es ergódico porque la matriz de transición contiene ceros" para `YOHAGOYOGAHOY`, `ANITALAVALATINA` y las vocales; sin embargo, **los tres grafos son irreducibles y aperiódicos** (verificado: P^k positiva para $k = 2$ en las vocales, $k = 6$ en `YOHAGO…`, $k = 4$ en `ANITALA…`), o sea que **sí** son ergódicos según el criterio de accesibilidad. En el parcial el docente no corrigió esa frase en las hojas que tenemos; si tu criterio de cátedra es "ceros ⇒ no ergódico", usá eso como respuesta de examen, pero sabé que el argumento correcto es el de accesibilidad entre estados. Un ejemplo de no ergódica de verdad: $P = \begin{bmatrix} 1 & 0 \\ 0.5 & 0.5 \end{bmatrix}$ (el estado 1 es absorbente: una vez que llegás no salís, y no se puede volver al 2).

5.3 Codificación eficiente de Markov (un código por estado)

Se construye un Huffman distinto para las transiciones desde cada estado. El primer símbolo se codifica con bloque fijo (define el estado inicial), y desde ahí se usa el diccionario del estado actual.

$$L_M(S_i) = \sum_j l(S_j) P(S_j|S_i), \qquad L_M = \sum_i P(S_i) L_M(S_i), \qquad H(S_M) \leq H(S)$$

Conocer el estado previo reduce la incertidumbre, por eso $H(S_M)$ (entropía de Markov) es menor o igual que la entropía de memoria nula $H(S)$ — y por eso L_M puede quedar por debajo de $H(S)$. Desventaja: hay que transmitir/conocer todos los diccionarios (uno por estado).

Entropía de la fuente de Markov (lo que piden como "entropía" en el bloque B):

$$H(S_M) = H(B|A) = \sum_i \pi_i \, H(\text{fila } i), \qquad H(\text{fila } i) = \sum_j P_{ij} \log_2 \frac{1}{P_{ij}}$$

Se pondera por la distribución **estacionaria** π (no por $1/q$).

Probabilidad de una cadena (ej. $p(eeii)$): $p(s_1 s_2 \ldots s_k) = \pi(s_1) \cdot P(s_2|s_1) \cdots P(s_k|s_{k-1})$. Ejemplo resuelto en 11.2.

Cómo se arma la matriz de transición desde la secuencia (fuente periódica con separadores, como en todos los parciales): contar **cuántas veces aparece cada par consecutivo** $s_i s_j$ y **dividir cada fila por la cantidad de veces que el estado s_i aparece como origen** (⚠️ no dividir por n: la matriz con "probabilidad conjunta" fue marcada "¿y Markov?" por el docente). Como la secuencia es cíclica

(`…|aei…`), el último símbolo transiciona al primero del bloque siguiente (**cierre circular**); así cada fila suma exactamente la cantidad de apariciones del estado. Todos los ejemplos resueltos de la cátedra están hechos así.

Receta de codificación de Markov de orden 1 ("codificar mediante Huffman, Fano y Shannon como fuente de Markov de orden 1") — ver el ejemplo completo en 11.2 y 11.3:

1. Matriz de transición y π.
2. Para **cada estado** (cada fila), tomar sus probabilidades **distintas de cero** y construir el código (Huffman / Fano / Shannon) *solo con ese conjunto de símbolos*. Una fila con **un único** sucesor ($p = 1$) usa **0 bits** (no hay nada que decidir).
3. El **primer símbolo** del mensaje se codifica con código de bloque (o con el código de la fuente independiente); a partir del segundo se usa el código de la fila del símbolo anterior.
4. $L_M = \sum_i \pi_i L_i$; eficiencia $\eta = H(S_M)/L_M$. Compararlo con el código de bloque y con el de fuente independiente.

5.4 Ejemplo resuelto (de cátedra) — secuencia de vocales AEEEEEEEOOIIAAOOOOEEEA

Con la matriz de transición dada, $H(S) = 1{,}8149$ bits/símbolo (memoria nula) pero $H(S_M) \approx 1{,}186 \leq H(S)$, $L_M \approx 1{,}299$ bits/símbolo (contra $L = 2$ de código bloque), y $\pi = (0{,}20;\ 0{,}45;\ 0{,}10;\ 0{,}25)$ para (A,E,I,O). Aprovechar la memoria de la fuente baja la longitud promedio en ≈35% respecto del código de bloque.

5.5 Ergodicidad — qué la rompe

No-ergodicidad: dependencia fuerte del estado inicial, estados inaccesibles desde otros (el sistema queda "atrapado": estados absorbentes o grupos de estados cerrados), ciclos estrictamente periódicos, y promedio temporal ≠ promedio estadístico. Test práctico: dibujar el grafo y ver si hay un estado al que no se puede llegar desde alguno, o uno desde el que no se puede salir. Ver la advertencia de 5.2 sobre la frase "tiene ceros ⇒ no ergódico".

5.6 Codificación predictiva (compuerta OREX / XOR) y búsqueda del K óptimo

Un predictor adivina el siguiente bit; la compuerta **OREX (XOR)** compara predicción y valor real y emite `1` **= error** y `0` **= acierto**. Si el predictor acierta casi siempre, la cadena queda **llena de ceros con unos aislados**, y eso se comprime con una **codificación por longitud de racha con contenedores de K bits**:

1. Para cada `1`, escribir en K bits la **cantidad de ceros consecutivos que lo preceden** (racha).
2. El valor máximo $2^K - 1$ (todos unos) significa **"la racha continúa"**: se emite ese contenedor y se sigue con otro contenedor para el resto de la racha. (Una racha de $m \geq 2^K - 1$ ceros usa $\lfloor m/(2^K - 1) \rfloor$ contenedores llenos más uno con el resto.)
3. Los **ceros finales** (después del último `1`) también se codifican con su propia racha.
4. Se prueba $K = 2, 3, 4, \ldots$ y se elige el que da **menos bits totales**; **se corta cuando $K+1$ empeora** respecto de K (más contenedores grandes ya no ayudan). ⚠️ El docente marcó: *"debió verificar que con K=4 aumenta la cantidad de bits"*.

Comprobación por fórmula (la pide el recuperatorio 2025): con p = probabilidad de **acierto** (cantidad de `0` sobre el total) la longitud promedio es $L(K) = \dfrac{K}{1 - p^{2^K - 1}}$ y el K que **minimiza** L es el óptimo; tiene que coincidir con el que sale de contar bits. (Cuidado con el exponente: es $2^K - 1$, el máximo de ceros que cabe en un contenedor, **no** 2^{K-1}; y p cuenta los ceros, no los unos.) Equivale a minimizar los bits por símbolo $K(1-p)/(1 - p^{2^K - 1})$.

Ejemplo (cadena OREX de los parciales de 2023) en 11.6; versiones 2025 con fórmula en 11.11 ($K_{opt} = 4$) y abajo.

Otra cadena de 2025 `000010000001100000000000100001` (30 bits; 5 unos; rachas $4, 6, 0, 11, 4$; $p = 25/30$): $K = 2$: 24 bits; $K = 3$**: 18 bits**; $K = 4$: 20; $K = 5$: 25; fórmula $L = 4{,}747$; $\mathbf{4{,}161}$; $4{,}278$; $5{,}018$ ⇒ $K_{opt} = \mathbf{3}$ (la hoja de un alumno usó $p = 5/25$ —la proporción de unos sobre un total mal contado— y el exponente 2^{K-1}, y por eso eligió $K = 2$).

---

6. Ejemplo integrador: comprimiendo `ABRACADABRA` con todos los métodos

Se usa siempre la misma fuente para poder comparar resultados en igualdad de condiciones. `ABRACADABRA` tiene **11 símbolos**, alfabeto $\{A, B, C, D, R\}$ ($q = 5$).

6.1 Punto de partida: frecuencias y entropía

Símbolo	Frecuencia	p_i	$I_i = -\log_2 p_i$
A	5	5/11 = 0,4545	1,1375 bits
B	2	2/11 = 0,1818	2,4594 bits
R	2	2/11 = 0,1818	2,4594 bits
C	1	1/11 = 0,0909	3,4594 bits

Símbolo	Frecuencia	p_i	$I_i = -\log_2 p_i$
D	1	1/11 = 0,0909	3,4594 bits

$$H(S) = \sum p_i I_i = \mathbf{2{,}0404}\ \text{bits/símbolo}$$

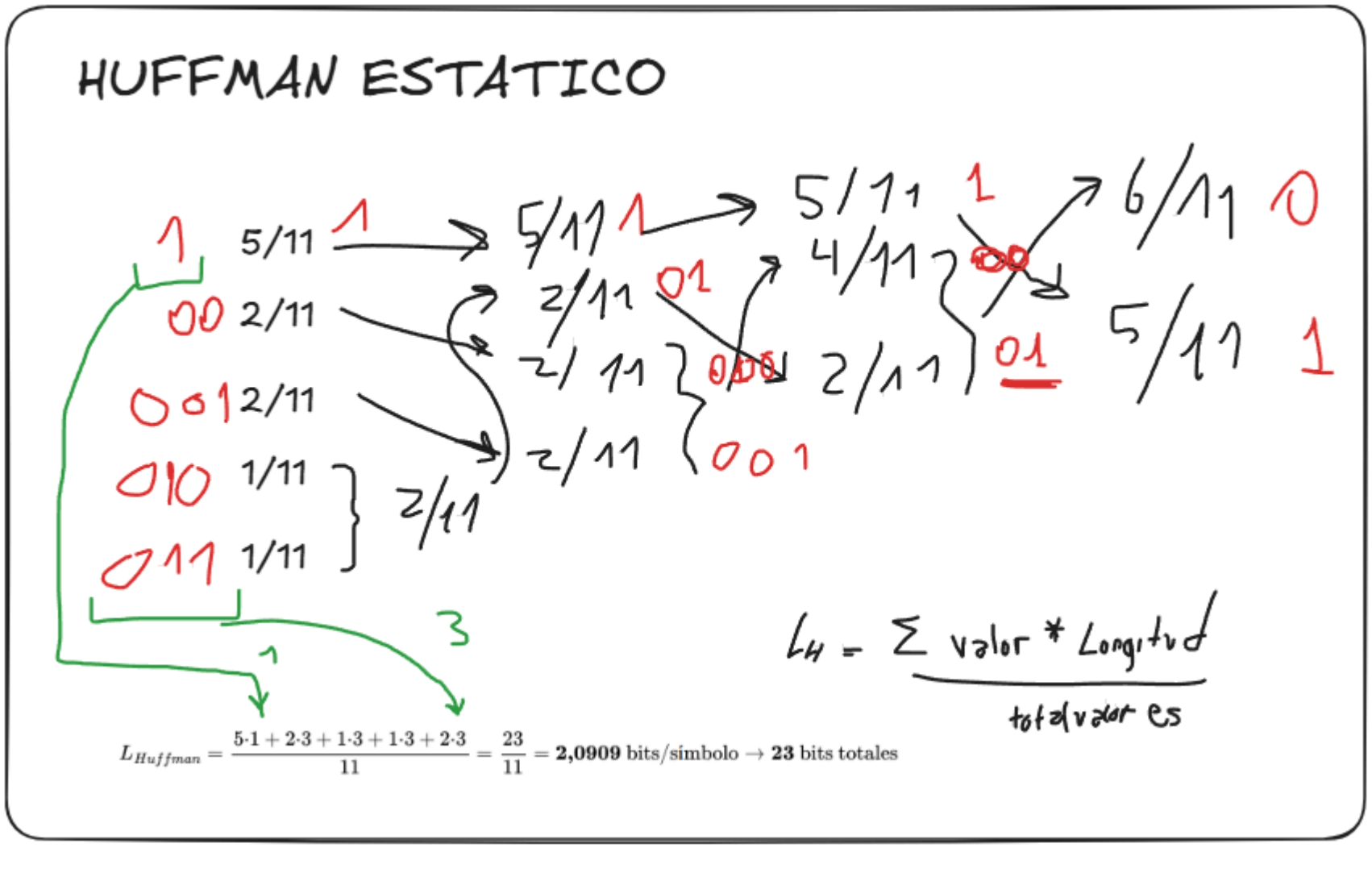
$$H_{max} = \log_2 5 = 2{,}3219, \qquad \eta = \frac{2{,}0404}{2{,}3219} = \mathbf{87{,}87\%}, \qquad R = \mathbf{12{,}13\%}$$

Cota teórica absoluta para comprimir esta secuencia completa: $H(S) \times 11 = \mathbf{22{,}44}$ bits. Ningún método sin pérdida puede bajar de ahí de manera sostenida — es la vara con la que se mide todo lo que sigue.

Código de bloque (longitud fija), como referencia de "no comprimir": $q = 5 \Rightarrow l = \lceil \log_2 5 \rceil = 3$ bits/símbolo → **33 bits** totales, $\eta = H/3 = 68{,}01\%$.

6.2 Huffman estático

Se ordenan los símbolos de mayor a menor probabilidad — **A, B, R, C, D** — y en cada paso se combinan los dos símbolos menos probables (C y D primero, ambos con $p = 1/11$) en un nodo cuya probabilidad es la suma, reordenando y repitiendo hasta llegar a un único nodo raíz.



Símbolo	p_i	Código	l_i
A	5/11	`0`	1
B	2/11	`111`	3
R	2/11	`110`	3
C	1/11	`101`	3
D	1/11	`100`	3

$$L_{Huffman} = \frac{5{\cdot}1 + 2{\cdot}3 + 2{\cdot}3 + 1{\cdot}3 + 1{\cdot}3}{11} = \frac{23}{11} = \mathbf{2{,}0909}\ \text{bits/símbolo} \rightarrow \mathbf{23}\ \text{bits totales}$$

$$\eta = \frac{H}{L} = \frac{2{,}0404}{2{,}0909} = \mathbf{97{,}58\%}$$

Frente al código de bloque (33 bits), Huffman ahorra 10 bits (30% menos) sobre esta misma palabra.

6.3 Shannon

$l_i = \lceil \log_2(1/p_i) \rceil$, y el código sale de tomar los primeros l_i bits de la función acumulada (FA) en binario, ordenando los símbolos de mayor a menor probabilidad:

SHANNON

Longitud Promedio

log2 11/5 <= A <= log2 11/5 + 1 -> 2

log2 11/2 <= B <= log2 11/2 + 1 -> 3

log2 11/2 <= R <= log2 11/2 + 1 -> 3

log2 11/1 <= C <= log2 11/1 + 1 -> 4

log2 11/1 <= D <= log2 11/1 + 1 -> 4

Simbolo	pi	FA	Dec	Bin	Long	Cod
A	5/11	0	0	0	2	0
B	2/11	5/11	0.45	0.011	3	011
R	2/11	7/11	0.64	0.101	3	101
C	1/11	9/11	0.81	0.1100	4	1100
D	1/11	10/11	0.90	0.1110	4	1110

$$L_{Shannon} = \frac{5{\cdot}2 + 2{\cdot}3 + 2{\cdot}3 + 1{\cdot}4 + 1{\cdot}4}{11} = \frac{30}{11} = \mathbf{2{,}7273} \rightarrow \mathbf{30}\ \text{bits},$$

$$L_{Shannon} = \frac{5{\cdot}2 + 2{\cdot}3 + 2{\cdot}3 + 1{\cdot}4 + 1{\cdot}4}{11} = \frac{30}{11} = \mathbf{2{,}7273} \rightarrow \mathbf{30}\ \text{bits}, \qquad \eta = \mathbf{74{,}81\%}$$

6.4 Fano

Se ordena por probabilidad decreciente — A, B, R, C, D — y se corta la lista en dos donde la suma de cada mitad se acerque más a 0,5, asignando 0 a un grupo y 1 al otro, y repitiendo recursivamente dentro de cada mitad hasta que cada grupo tenga un único símbolo.

FANO

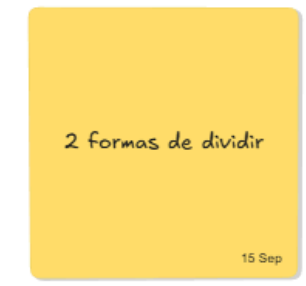
0 5/11 — 0.45 0

10 2/11 — 0.18 10 — 0.54 1

110 2/11 — 0.18 110 — 0.36 11

1110 1/11 — 0.09 1110 — 0.09 111

1111 1/11 — 0.09 1111



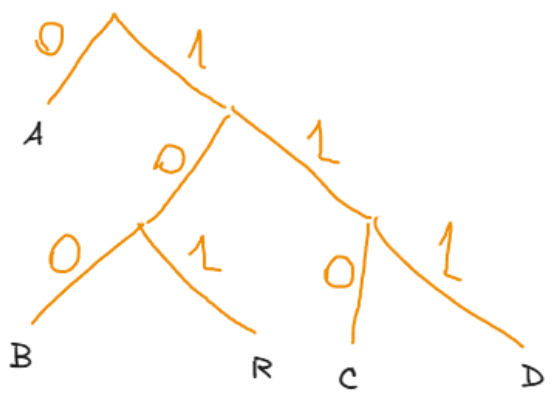
0 5/11 — 0.45 0 — 1°

100 2/11 — 100 — 0.54 1 — 3°

101 2/11 — 101 — 10 — 2°

110 1/11 — 110 — 11 — 4°

111 1/11 — 111



$$L_{Fano} = \frac{5{\cdot}1 + 2{\cdot}2 + 2{\cdot}3 + 1{\cdot}4 + 1{\cdot}4}{11} = \frac{23}{11} = \mathbf{2{,}0909} \rightarrow \mathbf{23}\ \text{bits}$$

Símbolo	Código	l_i
A	`0`	1
B	`10`	2
R	`110`	3
C	`1110`	4
D	`1111`	4

$$L_{Fano} = \frac{5{\cdot}1 + 2{\cdot}2 + 2{\cdot}3 + 1{\cdot}4 + 1{\cdot}4}{11} = \frac{23}{11} = \mathbf{2{,}0909} \rightarrow \mathbf{23}\ \text{bits}$$

Verificación de la relación teórica: $L_{Huffman}(2{,}0909) \leq L_{Fano}(2{,}0909) \leq L_{Shannon}(2{,}7273)$. Para esta fuente, Huffman y Fano **empatan exactamente** (ambos óptimos acá); Shannon queda claramente por detrás porque redondea las longitudes hacia arriba símbolo por símbolo sin margen de reacomodo.

6.5 Codificación aritmética

Se recorre el alfabeto una sola vez, en **el mismo orden decreciente de probabilidad usado en Shannon** (A, B, R, C, D — no alfabético), y se arma la tabla de intervalos acumulados en ese orden:

Símbolo	p_i	**Intervalo** $[Q, R)$
A	5/11	[0,0000 ; 0,4545)
B	2/11	[0,4545 ; 0,6364)
R	2/11	[0,6364 ; 0,8182)
C	1/11	[0,8182 ; 0,9091)
D	1/11	[0,9091 ; 1,0000)

Mecánica: en cada paso, $\text{nuevo_low} = low + rango \cdot Q_{símbolo}$, $\text{nuevo_high} = low + rango \cdot R_{símbolo}$, con $rango = high - low$. El intervalo se va achicando y "encerrando" cada vez más el número real que representa a toda la secuencia. Como el orden de los símbolos cambió respecto de una versión alfabética, **los intervalos de B, R, C y D también cambian** (no es solo cambiar el orden de la tabla: cada paso arrastra el nuevo $[Q, R)$ de cada símbolo).

Paso	**Símbolo**	**Intervalo resultante**
1	A	[0,00000000 ; 0,45454545)
2	B	[0,20661157 ; 0,28925620)
3	R	[0,25920361 ; 0,27422990)
4	A	[0,25920361 ; 0,26603374)
5	C	[0,26479190 ; 0,26541282)
6	A	[0,26479190 ; 0,26507414)
7	D	[0,26504848 ; 0,26507414)
8	A	[0,26504848 ; 0,26506014)
9	B	[0,26505378 ; 0,26505590)
10	R	[0,26505513 ; 0,26505551)
11	A	[0,26505513 ; 0,26505530)

Intervalo final exacto: $[0{,}26505513,\ 0{,}26505530)$. Cualquier número dentro de ese rango identifica sin ambigüedad la secuencia completa; el más corto es `0,2650552` (7 dígitos decimales).

> 💡 Nota sobre "3 dígitos decimales" (Tema 3 del examen modelo). **La consigna real es: trabajar con solo 3 decimales en TODOS los pasos** (cada cota se redondea a 3 decimales) **y codificar "hasta que no logren diferenciarse las cotas superior e inferior"**. Es decir, no se pide codificar todo el mensaje: se codifica hasta el símbolo anterior al que hace que $low = high$ con esa precisión; ahí el método **falla por falta de precisión** (el docente lo marcó así: *"SON IGUALES LAS COTAS SUPERIOR E INFERIOR. ERROR: NO SE PUEDE DISCERNIR UN VALOR ENTRE ELLO"*). El ejemplo completo del Tema 3 está en 11.5 (se codifican 4 símbolos y en el 5.º las cotas coinciden).
>
> Lo de arriba (intervalo con 8 decimales y número `0,2650552`) es la **versión de precisión completa**, útil para ver el método y la relación con la entropía (22,44 bits); la versión "3 decimales" es la que se pide en el examen. *(La versión anterior de este apunte decía que había que redondear solo al final: eso es la versión de precisión completa, no lo que pide el enunciado.)*

Tamaño equivalente en bits: el ancho del intervalo final es $\approx 1{,}75 \times 10^{-7}$, y $-\log_2(1{,}75 \times 10^{-7}) \approx \mathbf{22{,}44}$ bits — **coincide casi exactamente con la cota de entropía** $H(S) \times 11 = 22{,}44$ bits calculada en 6.1. Esta es la gran ventaja teórica de la codificación aritmética: al no estar atada a longitudes de código enteras por símbolo (como sí les pasa a Huffman/Shannon/Fano), se acerca arbitrariamente al límite de Shannon. En la práctica, emitiendo el número con algún margen de seguridad, el mensaje ocupa **≈23-24 bits**.

6.6 Transformada de Burrows-Wheeler (BWT)

BWT no comprime por sí misma: reordena la cadena para agrupar caracteres repetidos, dejándola lista para que RLE o Huffman compriman mejor después. Procedimiento:

1. Agregar un símbolo de fin de cadena, `$` (menor que cualquier letra): `ABRACADABRA$` (12 caracteres).
2. Generar las 12 rotaciones cíclicas.

3. Ordenarlas lexicográficamente.
4. Tomar la última columna.

Rotación ordenada	Último carácter
`$ABRACADABRA`	A
`A$ABRACADABR`	R
`ABRA$ABRACAD`	D
`ABRACADABRA$`	`$`
`ACADABRA$ABR`	R
`ADABRA$ABRAC`	C
`BRA$ABRACADA`	A
`BRACADABRA$A`	A
`CADABRA$ABRA`	A
`DABRA$ABRACA`	A
`RA$ABRACADAB`	B
`RACADABRA$AB`	B

Resultado: `ARD$RCAAAABB` (el `$` forma parte de la transformada; sin el `$`, para RLE, queda `ARDRCAAAABB`).

Se nota inmediatamente el efecto buscado: la cadena original no tenía ninguna repetición consecutiva, pero la transformada tiene una corrida `AAAA` (4 caracteres seguidos) y una `BB`. Aplicando RLE a `ARDRCAAAABB` se obtienen 7 "tokens" en vez de 11 símbolos sueltos: `A¹ R¹ D¹ R¹ C¹ A⁴ B²` — la ganancia de RLE crece con el largo de las corridas, así que en textos más largos (donde BWT agrupa corridas mucho más largas) el ahorro es mucho mayor que en este ejemplo de 11 letras.

BWT inversa (lo que pide el parcial 2022, `amnr$aaag`): se reconstruye la matriz completa **agregando la columna transformada a la izquierda de la tabla actual y reordenando, repetido n veces**; la fila que termina en `$` es la cadena original. Se verificó computacionalmente que la inversa de `ARD$RCAAAABB` reconstruye exactamente `ABRACADABRA$`. El ejercicio del parcial 2022 está resuelto paso a paso en 11.7 (`amnr$aaag` → `anagrama$`).

(Por qué BWT ayuda a la estadística: agrupa símbolos iguales, así que después de BWT un símbolo "predice" muy bien al siguiente; una matriz de transición de Markov armada sobre la transformada tiene valores cercanos a 1 y una entropía condicional mucho menor ⇒ códigos más cortos, más cerca de la entropía. Es la respuesta esperada a la teórica del Recuperatorio.)

6.7 Huffman dinámico (adaptativo)

A diferencia de Huffman estático, acá **no se conocen las frecuencias de antemano**: transmisor y receptor arrancan con un árbol de un único nodo, el **símbolo imaginario (NYT, "Not Yet Transmitted")**, y lo van construyendo símbolo a símbolo, **en espejo**, aplicando la **Sibling Property** (propiedad de hermandad) para reubicar nodos cuando un incremento de peso la rompe.

Receta (por cada símbolo que llega):

1. **Si el símbolo ya está en el árbol**: emitir su código actual (camino raíz → hoja, `0` = izquierda, `1` = derecha) y sumar 1 a su peso.
2. **Si es nuevo**: emitir el **código actual del NYT** seguido del **código fijo** del símbolo (con $q = 5$: A= `000`, B= `001`, R= `010`, C= `011`, D= `100`). Luego el NYT se "abre" en un nodo interno con dos hijos: un **NYT nuevo (izquierda)** y la **hoja del símbolo (derecha)** con peso 1.
3. **Actualizar pesos hacia arriba**: se suma 1 a la hoja y a todos sus ancestros hasta la raíz.
4. **Sibling Property (el "balanceo" del Excalidraw)**: numerando los nodos de **abajo hacia arriba y de izquierda a derecha**, los pesos deben quedar **no decrecientes**. Si al sumar 1 un nodo queda *más pesado* que uno que está *a su derecha/arriba* en esa numeración, se **intercambia ese nodo (con todo su subárbol) con el último nodo de su mismo peso** (que no sea su padre) **antes** de sumarle 1. El objetivo de la propiedad es **garantizar que el árbol siga siendo un árbol de Huffman óptimo** después de cada actualización. **Explicación completa, con el criterio de numeración y un ejemplo paso a paso, en 6.7.1.**

Resultado para `ABRACADABRA` (el que está **resuelto a mano en el Excalidraw** — y confirmado por una simulación independiente del algoritmo con estas mismas reglas, que reproduce el stream bit por bit):

Paso	Símbolo	¿Nuevo?	Código al emitir	Bits emitidos	Long.	Árbol después (hojas con su peso, código vigente)
1	A	Sí	NYT vacío + fijo	`000`	3	A= `1`, NYT= `0`
2	B	Sí	NYT= `0` + `001`	`0001`	4	A= `1`, B= `01`, NYT= `00`
3	R	Sí	NYT= `00` + `010`	`00010`	5	**swap**: A pasa a `0`; B= `11`, R= `101`, NYT= `100`
4	A	No	A= `0`	`0`	1	A pesa 2

Paso	Símbolo	¿Nuevo?	Código al emitir	Bits emitidos	Long.	Árbol después (hojas con su peso, código vigente)
5	C	Sí	NYT=`100` + `011`	`100011`	6	**swap**: B sube a la izquierda: A=`0`, B=`10`, R=`111`, C=`1101`, NYT=`1100`
6	A	No	A=`0`	`0`	1	A pesa 3
7	D	Sí	NYT=`1100` + `100`	`1100100`	7	**swap**: B=`110`, R=`111`, C=`101`, D=`1001`, NYT=`1000`
8	A	No	A=`0`	`0`	1	A pesa 4
9	B	No	B=`110`	`110`	3	**swap** B↔R: B=`111`, R=`110`
10	R	No	R=`110`	`110`	3	R y B pesan 2
11	A	No	A=`0`	`0`	1	A pesa 5

$$\text{Stream total: } \underbrace{000}_{A}\ \underbrace{0001}_{B}\ \underbrace{00010}_{R}\ \underbrace{0}_{A}\ \underbrace{100011}_{C}\ \underbrace{0}_{A}\ \underbrace{1100100}_{D}\ \underbrace{0}_{A}\ \underbrace{110}_{B}\ \underbrace{110}_{R}\ \underbrace{0}_{A} = \mathbf{35}\text{ bits}$$

Cuenta de bits: $3+4+5+1+6+1+7+1+3+3+1=35$. Árbol final (pesos): `D1 C1 R2 B2 → 2 4 · A5 · 6 · 11`, es decir raíz 11 = A_5 (código `0`) + nodo 6; el nodo 6 = nodo 2 (hojas D_1 y C_1) + nodo 4 (hojas R_2 y B_2). Promedio: $35/11 \approx 3{,}18$ bits/símbolo.

> ✅ **Corrección respecto de versiones anteriores del apunte**: antes decía 32 bits (`…00000010001011001101000100111001`), resultado de una simulación con otra convención de desempate. **El resultado válido es el de la resolución manual: 35 bits**, que además coincide con una implementación del algoritmo con las reglas de arriba (swap con el último nodo del mismo peso). Si en el examen resolvés a mano siguiendo el árbol paso a paso con la Sibling Property, tenés que llegar a este stream.

Se ve el patrón esperado: los 5 símbolos **nuevos** (A, B, R, C, D, la primera vez que aparecen) cuestan 3 a 7 bits (camino al NYT + código fijo de 3 bits), mientras que las repeticiones de A, B y R se emiten en 1 o 3 bits. Con solo 11 símbolos el "costo de aprendizaje" todavía pesa mucho: $35/11 \approx 3{,}18$ bits/símbolo, **peor incluso que el código de bloque** (33 bits) y que Huffman estático (23). Esto es normal y es justamente la desventaja conocida de los métodos adaptativos en mensajes cortos: **no hay tabla que transmitir de entrada, pero el aprendizaje sobre la marcha cuesta bits al principio**. Con un mensaje mucho más largo, el Huffman dinámico converge al mismo rendimiento que el estático (y lo supera en la práctica porque el estático necesita transmitir su tabla de códigos como cabecera).

> ⚠️ El árbol exacto en cada paso (y por lo tanto los bits exactos) depende de la **convención de desempate** (a qué nodo del mismo peso se intercambia, de qué lado va el NYT) y el enunciado/docente la fija. Lo que **no** cambia es: (a) el mecanismo (NYT, sibling property, actualización en espejo), (b) que el stream generado siempre se puede decodificar sin ambigüedad siguiendo exactamente el mismo algoritmo del lado receptor, y (c) que **el árbol de Huffman dinámico y el estático no tienen por qué coincidir** (pregunta teórica fija; ejemplo: símbolos A, C, S → estático `A=1, C=00, S=01`, dinámico `A=0, S=10, C=11`).

Decodificación: el receptor hace lo mismo en espejo. Lee bit a bit bajando por su árbol hasta una hoja; si llega al **NYT**, lee los $\lceil \log_2 q \rceil$ bits siguientes como código fijo del símbolo nuevo; luego actualiza su árbol exactamente igual que el emisor. Ejemplo resuelto en 11.8 (`000010010010011010` → `ANDABAN`).

6.7.1 La Sibling Property en detalle: qué es, para qué sirve, cuándo se aplica y cómo se comprueba

El Tema 2 del parcial pide literalmente: *"identificar en qué paso(s) se aplica la Sibling Property y cuál es el objetivo de que el árbol la cumpla"*. Son **dos respuestas distintas**: (1) *en qué pasos* y (2) *para qué*. Acá están las dos, con el procedimiento para poder hacerlo a mano.

Definición (Gallager, 1978). Un árbol binario con pesos cumple la *sibling property* (propiedad de hermandad) si:

1. todo nodo interno tiene **dos hijos** (todo nodo salvo la raíz tiene un hermano), y
2. los $2m-1$ nodos se pueden **numerar** $1, 2, \ldots, 2m-1$ de modo que los pesos sean **no decrecientes** $w_1 \le w_2 \le \cdots \le w_{2m-1}$, con los hermanos en números consecutivos y cada padre con un número mayor que el de sus hijos.

Objetivo (la respuesta a "¿para qué?"). Teorema de Gallager: *un árbol binario con pesos en las hojas es un árbol de Huffman **si y solo si** cumple la sibling property*. Por eso, mantener la propiedad después de **cada** símbolo garantiza que el árbol actual **es un árbol de Huffman —óptimo— para las frecuencias vistas hasta ese momento**: el símbolo más frecuente siempre tiene el código más corto, y se logra **sin reconstruir el árbol desde cero** (un intercambio de nodos cuesta mucho menos que rearmar todo). Como emisor y receptor aplican la misma regla en espejo, los dos árboles se mantienen idénticos sin transmitir ninguna tabla.

Cómo se numera (esto es lo que hay que dibujar en el examen). Nivel por nivel, **de abajo hacia arriba**, y dentro de cada nivel **de izquierda a derecha**. El NYT (peso 0) queda con el número 1 y la raíz con el último. Escribí los pesos en ese orden: la lista **nunca puede bajar**. Esa lista es tu "chequeo": si en algún lado baja, falta un swap o está mal hecho.

Cuándo se rompe y qué se hace. Al procesar un símbolo se suma 1 a la hoja y después a cada ancestro hasta la raíz. Para **cada nodo q de esa cadena, antes de sumarle 1**:

1. Buscar el nodo de **mayor número** que tenga **el mismo peso** que q (el "líder" de su bloque de pesos iguales).
2. Si el líder es q mismo o es el **padre** de q → no hay nada que hacer.
3. Si no → **intercambiar** q **(con todo su subárbol) con el líder.** Los pesos viajan con los nodos; solo cambian de lugar (y por lo tanto de código).
4. Recién ahí sumar 1 a q y pasar a su padre.

La razón del paso 3: si se le sumara 1 a q sin moverlo, quedaría **más pesado que un nodo numerado después** de él (la lista de pesos bajaría) y el árbol dejaría de ser de Huffman. Intercambiando con el último nodo de su mismo peso, el que sube de peso queda a la derecha de todos los de peso menor o igual.

Dónde se dispara en la práctica (esto responde "en qué pasos"):

- **Símbolo repetido** cuya hoja tiene el mismo peso que otras hojas numeradas más a la derecha (ej.: la `E` del paso 4 del ejemplo de abajo; la `B` del paso 9 de `ABRACADABRA`).
- **Símbolo nuevo**: después de abrir el NYT, los pesos suben desde la hoja nueva y el nodo que contiene al NYT suele toparse con una hoja de peso 1 a su derecha (pasos 3, 5 y 7 de `ABRACADABRA`; pasos 3 y 5 del ejemplo de abajo).
- **No se dispara** cuando el nodo ya es el líder de su bloque (no hay nadie con su mismo peso más a la derecha): por ejemplo los pasos 1, 2, 4, 6, 8, 10 y 11 de `ABRACADABRA`. **Los 4 pasos con balanceo de `ABRACADABRA` son el 3, el 5, el 7 y el 9** (los cuatro del Excalidraw).

Variantes. El procedimiento de arriba es el de **FGK**. El algoritmo de **Vitter** agrega que, entre nodos de igual peso, las **hojas** deben numerarse antes que los nodos internos (minimiza la longitud máxima del código). Si el enunciado no nombra a Vitter, usá FGK; los desempates pueden dar árboles distintos, pero **todos son de Huffman** (misma longitud media).

Ejemplo guiado — Tema 2 (2023), primeros 5 caracteres: `a e i e o`. Código fijo $q = 5$: a= `000`, e= `001`, i= `010`, o= `011`, u= `100`. En la columna "numeración" se lista **(nodo: peso)** de abajo hacia arriba y de izquierda a derecha; n, m, p son nodos internos.

Paso	Símbolo	Bits emitidos	¿Qué pasa con la Sibling Property?	Árbol después (código vigente)	Numeración (nodo: peso)
1	a (nuevo)	`000` (NYT vacío + fijo)	**No hace falta**: no hay nadie con quien romper el orden.	a= `1`, NYT= `0`	NYT:0, a:1, raíz:1
2	e (nuevo)	`0` + `001` = `0001`	**No hace falta**: el nodo n que sube a 1 queda a la izquierda de a (peso 1), orden respetado.	a= `1`, e= `01`, NYT= `00`	NYT:0, e:1, n:1, a:1, raíz:2
3	i (nuevo)	`00` + `010` = `00010`	⚠️ **Se aplica.** Al abrir el NYT aparece m (peso 0→1) y hay que subir n de 1 a 2, pero **a** (peso 1) está numerado después de n ⇒ quedaría $w_n = 2 > w_a = 1$. Líder de peso 1 = a ⇒ **swap** $n \leftrightarrow$ **a**.	a= `0`, e= `11`, i= `101`, NYT= `100`	NYT:0, i:1, m:1, e:1, a:1, n:2, raíz:3
4	e (repetido)	`11`	⚠️ **Se aplica.** e (peso 1, nº 4) iba a pasar a 2, pero el líder de peso 1 es **a** (nº 5) ⇒ **swap e** $\leftrightarrow$ **a**; e sube a hijo de la raíz.	e= `0`, a= `11`, i= `101`, NYT= `100`	NYT:0, i:1, m:1, a:1, e:2, n:2, raíz:4
5	o (nuevo)	`100` + `011` = `100011`	⚠️ **Se aplica.** Se abre el NYT (p), y al subir m de 1 a 2 el líder de peso 1 es **a** ⇒ **swap** $m \leftrightarrow$ **a** (a pasa a la izquierda de n). Después n (peso 2) ya es el líder de su bloque ⇒ sin swap.	e= `0`, a= `10`, i= `111`, o= `1101`, NYT= `1100`	NYT:0, o:1, p:1, i:1, a:1, m:2, e:2, n:3, raíz:5

Cada fila termina con una lista **no decreciente ✓**. Flujo emitido: `000 0001 00010 11 100011` $= 3 + 4 + 5 + 2 + 6 = \mathbf{20}$ bits.

Respuesta para el parcial (para copiar): *"La Sibling Property se aplica en los pasos 3, 4 y 5 (en el 1 y el 2 el orden ya se cumple). Su objetivo es que, tras cada actualización de pesos, el árbol siga siendo un árbol de Huffman —el más corto para el símbolo más frecuente — sin reconstruirlo desde cero, manteniendo sincronizados emisor y receptor."*

Comparación con el código fijo (inciso 4 del Tema 2): código de igual longitud $\lceil \log_2 5 \rceil = 3$ bits ⇒ $5 \cdot 3 = 15$ bits; Huffman dinámico $= 20$ bits ⇒ compresión $= 1 - 20/15 = \mathbf{-33{,}3\%}$ (**expande**). Es normal con solo 5 símbolos: 4 son nuevos (cada uno paga camino al NYT + 3 bits fijos) y solo la `e` repetida se emite barata (2 bits). El método se paga a lo largo de mensajes más largos (ver el comentario de `ABRACADABRA`). *(Si el docente interpreta "primeros 5 caracteres" de otra forma, el procedimiento es el mismo; lo que cambia son los símbolos.)*

6.8 PPM-C (Prediction by Partial Matching)

Es el método más potente de los que entran en el parcial, y el que cierra la unidad 5. Tiene **dos etapas separadas**: primero un modelo estadístico por contexto (PPM-C en sí) decide probabilidades; después un codificador aritmético convierte esas probabilidades en bits.

Idea: para predecir el próximo símbolo, se mira el contexto (los n caracteres anteriores), empezando por el orden más alto disponible (acá, orden 2) y bajando de a uno si el símbolo no apareció nunca en ese contexto, hasta llegar a un orden -1 que es una distribución uniforme

sobre un alfabeto base (en el ejemplo de cátedra, 64 símbolos → $1/64$ cada uno, representando un alfabeto genérico de texto, no solo las 5 letras de esta palabra).

El escape (ESC): cuando el contexto actual no conoció nunca el símbolo, se emite un evento especial ESC y se baja de orden. En PPM-C el "peso" del ESC no es fijo: $f(\mathrm{ESC}) = U$, donde U es la cantidad de símbolos *distintos* ya vistos en ese contexto — cuantos más símbolos diferentes conoce un contexto, más probable se considera que aparezca uno todavía no visto.

$$P(ESC) = \frac{U}{total + U}, \qquad P(s|\text{contexto}) = \frac{freq(s)}{total + U}$$

Etapa 1 — modelado (contextos de orden 2, 1, 0 y -1 para cada letra, actualizando tablas después de cada símbolo):

Paso	Símbolo	Búsqueda de contexto	Evento codificado
1	A	O2, O1, O0 vacíos → ESC·ESC·ESC	A en O−1 (1/64)
2	B	O2, O1 vacíos, O0 no tiene B → ESC·ESC·ESC(O0=1/2)	B en O−1 (1/64)
3	R	ídem, O0 no tiene R → ESC·ESC·ESC(O0=1/2)	R en O−1 (1/64)
4	A	O2, O1 vacíos, **O0 sí tiene A** → ESC·ESC	A en O0 (1/6)
5	C	O2(AC) vacío, O1(A) no tiene C, O0 no tiene C → ESC·ESC·ESC	C en O−1 (1/64)
6	A	O2, O1 vacíos, O0 sí tiene A → ESC·ESC	A en O0 (2/9)
7	D	O2(CA) vacío, O1(A) no tiene D, O0 no tiene D → ESC·ESC·ESC	D en O−1 (1/64)
8	A	O2, O1 vacíos (AD), O0 sí tiene A → ESC·ESC	A en O0 (1/4)
9	B	O2(DA) vacío, **O1(A) sí tiene B** → ESC	B en O1 (1/6)
10	R	**O2(AB) sí tiene R** → codifica directo	R en O2 (1/2)
11	A	**O2(BR) sí tiene A** → codifica directo	A en O2 (1/2)

El patrón clave: al principio todo son escapes hasta O−1 (nada de contexto conocido todavía); a mitad de camino el contexto de orden 0 ya predice bien las repeticiones de A; y hacia el final, con la secuencia "AB" y "BR" ya vistas, el orden 2 predice directamente R y A **sin ningún escape** — el modelo aprendió la estructura local de la palabra.

Etapa 2 — codificador aritmético: toma cada evento (ESC o símbolo) con su probabilidad y va angostando un único intervalo $[low, high)$, igual que en 6.5, pero encadenando varias probabilidades por símbolo (una por cada ESC que hubo que atravesar). Usa 3 reglas de renormalización para no perder precisión:

- **E1** ($high \leq 0{,}5$): todo el intervalo empieza con 0 → emitir 0, duplicar.
- **E2** ($low \geq 0{,}5$): todo el intervalo empieza con 1 → emitir 1, duplicar.
- **E3** ($0{,}25 \leq low < 0{,}5 < high \leq 0{,}75$): todavía ambiguo → no emitir, guardar un "bit pendiente" y seguir angostando; cuando después aparece E1 o E2, se resuelve emitiendo ese bit y luego los pendientes invertidos.

Bits emitidos por paso (contando los pendientes resueltos):

Paso	Símbolo	Bits en este paso
1	A	000000 (6)
2	B	0000001 (7)
3	R	0010001 (7)
4	A	10 (2)
5	C	00000001 (8)
6	A	— (2 bits pendientes)
7	D	01100101111 (11, incluye los 2 pendientes)
8	A	— (1 bit pendiente)
9	B	1001 (4)
10	R	— (esperando)
11	A	1 (1)
Flush	—	011 (3, cierre final)

Total: **49** bits

Comparación (misma base de 64 símbolos que usa el ejemplo): sin modelado, 11 símbolos × 6 bits (para indexar 1 de 64) = 66 bits. PPM-C logra **49 bits → 25,8% de reducción**, a pesar de que la palabra es corta y el modelo recién está aprendiendo.

> ⚠️ Estos 49 bits **no son comparables número contra número** con los 23-33 bits de Huffman/Shannon/Fano/aritmética de la sección anterior: aquellos métodos asumen un alfabeto cerrado y conocido de antemano de solo 5 símbolos, mientras que PPM-C usa un alfabeto base de 64 símbolos (más realista para texto real, donde no se sabe de antemano qué letras van a aparecer). La comparación justa de PPM-C es contra el código de bloque *de su propio alfabeto* (66 bits) — ahí sí se ve la ganancia real del modelado por contexto.

6.9 Lempel-Ziv (ventana deslizante, LZ77)

Es el método pedido en la variante Tema 4 del examen modelo ("LZ con ventana deslizante, ventana de 8 caracteres"). A diferencia de todo lo anterior, acá **no hay tabla de probabilidades**: el "diccionario" es directamente la parte del mensaje ya procesada (el *buffer de búsqueda*), y en cada paso se busca ahí la coincidencia más larga con lo que sigue (el *buffer de mirar más allá*). Se emite una tupla **(desplazamiento, longitud, símbolo siguiente)**.

Paso	Buffer de búsqueda (diccionario previo)	Buffer de mirar más allá (a procesar)	Coincidencia hacia atrás	Salida (marca generada)
1	(vacío)	ABRACADABRA	No hay, sig. A	`(0,0,'A')`
2	A	BRACADABRA	No hay, sig. B	`(0,0,'B')`
3	AB	RACADABRA	No hay, sig. R	`(0,0,'R')`
4	ABR	ACADABRA	Hay coinc. para "A". Está a 2 pos., long. 1. Sig. C	`(2,1,'C')`
5	ABRAC	ADABRA	Hay coinc. para "A". Está a 4 pos., long. 1	`(4,1,'D')`
6	ABRACAD	ABRA	Hay coinc. para "ABRA". Está a 6 pos., long. 4	`(6,4,'eof')`

Salida final: 6 tuplas en vez de 11 símbolos sueltos — los pasos 4, 5 y 6 ya no gastan una marca por letra, sino que "apuntan hacia atrás" y reaprovechan lo ya visto (el paso 6 reconstruye de una sola vez las 4 letras que faltan, `ABRA`). Si se acotan los campos a la ventana de 8 caracteres pedida (desplazamiento y longitud en 4 bits cada uno) más 3 bits para el símbolo (alfabeto de 5 letras + marca de fin), cada tupla ocupa ≈11 bits $\rightarrow 6 \times 11 \approx \mathbf{66}$ bits (cifra solo ilustrativa: el ancho exacto de cada campo lo fija el enunciado). Para un mensaje tan corto, el costo de emitir desplazamiento+longitud en cada tupla todavía no se paga solo; la ventaja de LZ77 aparece cuando el texto es lo bastante largo como para que las coincidencias sean mucho más largas que 1 o 4 caracteres.

6.10 Lempel-Ziv-Welch (LZW)

A diferencia de LZ77, LZW no emite desplazamiento+longitud: mantiene un **diccionario que arranca con el alfabeto base ya cargado** (acá, con códigos tipo ASCII: A=65, B=66, C=67, D=68, R=82) y le va agregando cada nueva secuencia de dos símbolos que encuentra, emitiendo siempre el índice de la cadena más larga ya conocida en el diccionario.

Paso (lectura)	Cadena acumulada (I)	Evaluación (I+x)	Salida (código emitido)	Nueva entrada al diccionario
1 (lee 'A')	A	Lee B, "AB" no está en el dicc.	65 (índice de A)	256 = AB
2 (lee 'B')	B	Lee R, "BR" no está	66 (índice de B)	257 = BR
3 (lee 'R')	R	Lee A, "RA" no está	82 (índice de R)	258 = RA
4 (lee 'A')	A	Lee C, "AC" no está	65 (índice de A)	259 = AC
5 (lee 'C')	C	Lee A, "CA" no está	67 (índice de C)	260 = CA
6 (lee 'A')	A	Lee D, "AD" no está	65 (índice de A)	261 = AD
7 (lee 'D')	D	Lee A, "DA" no está	68 (índice de D)	262 = DA
8 (lee 'A')	A	Lee B, "AB" **sí está** (256)	— (sigue acumulando)	—
9 (lee 'B')	AB	Lee R, "ABR" no está	256 (índice de AB)	263 = ABR
10 (lee 'R')	R	Lee A, "RA" **sí está** (258)	— (sigue acumulando)	—
11 (lee 'A')	RA	fin de cadena	258 (índice de RA)	EOF

Salida final: **9 códigos** (`65, 66, 82, 65, 67, 65, 68, 256, 258`) en vez de 11 símbolos — el diccionario llegó a capturar "AB" y "RA" como unidades de 2 caracteres cada una (pasos 8 y 10, donde no se emite nada porque sigue "buscando algo más largo"). Con un ancho fijo de 9 bits por código (alcanza para llegar hasta el índice 263), el mensaje ocupa $9 \times 9 = \mathbf{81}$ bits: más que el propio código de bloque (33 bits), otra vez porque en un mensaje tan corto el diccionario recién está arrancando. La ganancia real de LZW (igual que la de LZ77) aparece en archivos largos, donde las entradas del diccionario terminan representando frases enteras que se repiten muchas veces.

---

7. Tabla comparativa final — ¿cuál comprimió mejor?

Grupo A — alfabeto cerrado y conocido de antemano (5 símbolos: A,B,C,D,R)

Método	Bits totales	Bits/símbolo	$\eta = H/L$	Comentario
Código de bloque	33	3,000	68,0%	Referencia, sin comprimir
Shannon	30	2,727	74,8%	Redondeo símbolo a símbolo, el más simple pero el menos eficiente de los estadísticos
Huffman dinámico	35*	3,182*	64,1%*	*Resuelto a mano (Excalidraw). Incluye el costo de "aprender" el alfabeto: en 11 símbolos queda peor que el bloque (33); converge a Huffman estático con mensajes largos
Huffman estático	23	2,091	97,6%	Óptimo entre los códigos de longitud entera
Fano	23	2,091	97,6%	Empata con Huffman en esta fuente particular
Aritmética	≈23-24	≈2,1	≈93-96%	Prácticamente pegada a la cota teórica $H \cdot n = 22{,}44$ bits
Cota teórica ($H \times 11$)	*22,44*	*2,040*	*100%*	*Límite inalcanzable en la práctica, pero se puede rozar*

Grupo B — alfabeto general de 64 símbolos (más realista para texto)

Método	Bits totales	Reducción vs. bloque
Código de bloque (6 bits/símbolo)	66	—
PPM-C + aritmética	49	25,8%

Grupo C — métodos de diccionario (no usan tabla de probabilidades, sino coincidencias/índices)

Método	Unidades emitidas	Bits totales (ilustrativo)*	Comentario
LZ77 (ventana deslizante)	6 tuplas (desplaz., long., símbolo)	≈66	Empieza a "apuntar hacia atrás" recién en el paso 4
LZW	9 códigos de diccionario	≈81	Ya reutiliza "AB" y "RA" como unidades completas

*El bit-total de estos dos depende del ancho de campo que fije el enunciado (tamaño de ventana, tamaño de diccionario); son valores ilustrativos, no una cota cerrada como en Huffman/Shannon/Fano.

Conclusiones que hay que poder explicar en el parcial:

1. **Huffman ≈ Fano ≤ Aritmética ≤ cota de entropía**, siempre en ese orden o mejor (la aritmética nunca puede ser peor que Huffman en el límite, porque no está atada a longitudes enteras).
2. **Shannon es el más simple de calcular pero el menos eficiente** de los tres estadísticos clásicos — se nota en la tabla: 30 bits contra 23 de Huffman/Fano para la misma fuente.
3. Los métodos **adaptativos y de diccionario (Huffman dinámico, PPM-C, LZ77, LZW) pagan un costo de aprendizaje** al principio del mensaje — por eso en textos cortos como este pueden rendir peor en bits/símbolo que un Huffman estático calculado con las frecuencias ya conocidas. Su ventaja real aparece en mensajes largos, donde además **no hace falta transmitir ninguna tabla/cabecera** (el estático sí la necesita, lo cual en archivos chicos puede comerse toda la ganancia).
4. **BWT no compite en esta tabla** porque no es, por sí misma, un compresor: es un paso de *reordenamiento* que expone corridas repetidas para que RLE o Huffman las aprovechen después. Su beneficio crece con la longitud y la redundancia del texto, no se ve bien en una palabra de 11 letras.
5. **PPM-C es el método más potente en términos absolutos** (es el que efectivamente se usa en compresores reales de texto), pero para juzgarlo hay que compararlo contra la referencia de *su propio* alfabeto base, no contra un alfabeto reducido armado a medida de la palabra.
6. **LZ77 vs. LZW**: LZ77 mira hacia atrás en el propio texto ya leído (sin diccionario aparte, pero repite desplazamiento+longitud en cada tupla); LZW arma un diccionario explícito de frases que solo crece (nunca repite el mismo dato dos veces, pero necesita que ese diccionario tenga tiempo de poblarse). Por eso el orden recomendado en la pregunta teórica cerrada #1 (LZW antes que Huffman)

tiene sentido: LZW saca la redundancia de "frases repetidas", y recién sobre ese resultado ya comprimido Huffman aprovecha lo que quede de asimetría en las frecuencias.

---

8. Chuleta de fórmulas

Concepto	Fórmula
Información	$I_i = -\log_2 p_i$
Entropía	$H = -\sum p_i \log_2 p_i$
Entropía máxima	$H_{max} = \log_2 q$
Rendimiento / redundancia	$\eta = H/H_{max}$, $R = 1 - \eta$
Prob. de salida	$P(b_j) = \sum_i P(a_i)P(b_j\|a_i)$
Bayes (canal)	$P(a_i\|b_j) = \dfrac{P(a_i)P(b_j\|a_i)}{P(b_j)}$
Información mutua	$I(A;B) = H(A) - H(A\|B) = H(B) - H(B\|A)$
Capacidad	$C = \max_{P(a)} I(A;B)$
Capacidad BSC	$C_{BSC} = 1 - H(p)$
Canal asimétrico	$I(A;B) = H(B) - H(B\|A)$ — **no** usar $1 - H(p)$
Kraft	$\sum r^{-l_i} \leq 1$
Longitud promedio	$L = \sum p_i l_i$
Eficiencia de código	$\eta = H/L$
Primer Teorema de Shannon	$H \leq L < H + 1$
Shannon (longitudes)	$l_i = \lceil \log_2(1/p_i) \rceil$
Nyquist	$f_s \geq 2B$
Cuantificación	$\Delta = (V_{max} - V_{min})/L$, $L = 2^n$
Tasa de bits	$R = f_s \cdot n \cdot c$
Extensión de orden n	$H(S^n) = nH(S)$, $L_n/n \to H(S)$
Markov: long. promedio	$L_M = \sum_i P(S_i)L_M(S_i)$
Markov: entropía	$H(S_M) \leq H(S)$
Distribución estacionaria	$\pi P = \pi$, $\sum \pi_i = 1$
Aritmética: intervalo	$newLow = low + rango \cdot Q_s$, $newHigh = low + rango \cdot R_s$
Aritmética: bits necesarios	$\lceil -\log_2 \prod p_i \rceil$ (ancho del intervalo final $= \prod p_i$)
Entropía en base r	$H_r = H_2/\log_2 r$; $R = 1 - H_r/\log_r q$
Compresión vs. longitud fija	$1 - L/L_{fijo}$, $L_{fijo} = \lceil \log_2 q \rceil$
Markov: entropía de la fuente	$H(S_M) = \sum_i \pi_i H(\text{fila } i)$
Prob. de una cadena (Markov 1)	$p(s_1 \dots s_k) = \pi(s_1) \prod P(s_{t+1}\|s_t)$
Matriz de transición	$P_{ij} = \dfrac{\#(s_i s_j)}{\#(s_i \text{ como origen})}$ (fila suma 1)
Entropías del canal	$H(A,B) = H(A) + H(B\|A)$; $H(A\|B) = H(A,B) - H(B)$
Capacidad BSC / mejor canal	$C = 1 - H(p)$ / $C = \log_2 q$ (identidad)
Capacidad canal simétrico $q \times q$	$C = \log_2 q - H(\text{fila})$ (entrada equiprobable)
Discretización con valor medio	ancho $= 2e$; niveles $= \text{rango}/(2e)$; $n = \lceil \log_2 \text{niveles} \rceil$
Predictiva (rachas)	bits $= \#\text{contenedores} \cdot K$; elegir K mínimo, cortar cuando $K+1$ empeora
Vigenère	cifrar $c = (m + k) \bmod 26$; descifrar $m = (c - k) \bmod 26$ ($A = 0, \dots, Z = 25$)
PPM-C: probabilidades	$P(ESC) = \dfrac{U}{total + U}$, $P(s) = \dfrac{freq(s)}{total + U}$

---

9. Preguntas teóricas cerradas (con respuesta breve)

1. **¿LZW antes o después de Huffman?** → Primero LZW, después Huffman: LZW saca redundancia de largo alcance (crea el diccionario), y sobre ese texto ya más "plano" Huffman asigna códigos cortos a lo frecuente.

2. **Fuente de 7 símbolos donde Huffman=Shannon=Fano** → cuando $p_i = r^{-l_i}$ exactamente para todos los símbolos (ej. $1/2, 1/4, 1/8, 1/16, 1/32, 1/64, 1/64$): Kraft=1, H=L. Es el caso **ideal** de la codificación de fuente: si las probabilidades son potencias exactas de la base r, la longitud óptima $l_i = \log_r(1/p_i)$ ya da un número entero, sin nada para redondear. Por eso los tres métodos —que en general dan resultados distintos— coinciden acá: Shannon no tiene nada que redondear hacia arriba ($\lceil \cdot \rceil$ no cambia nada), Huffman arma un árbol óptimo que resulta tener exactamente esas longitudes, y Fano divide el alfabeto en mitades perfectamente equilibradas. Kraft=1 confirma que el árbol de prefijos queda completamente lleno (no sobra ningún código), y H=L es la definición misma de **código compacto** ($\eta = 100\%$, $R = 0\%$): no hay ningún margen de mejora teórica posible. Fuera de este caso especial (como en ABRACADABRA, donde las probabilidades no son potencias de 2), siempre queda algo de "pérdida por redondeo", y ahí aparece la desigualdad estricta $H \leq L < H + 1$.
3. **¿Para qué sirve la compansión? (necesidad y fenómeno)** → es compresión + expansión: se comprime la dinámica con una curva no lineal antes de cuantificar y se expande en el receptor (cuantificación no uniforme: más resolución para amplitudes chicas, que son las más frecuentes). Necesidad: eficiencia en almacenamiento y transmisión, ahorro de costos (ver 3.6).
4. **¿Instantáneo implica unívoco, o al revés?** → todo instantáneo es unívoco; no toda secuencia unívoca es instantánea. Huffman siempre produce código instantáneo.
5. **¿Los árboles de Huffman estático y dinámico coinciden siempre? (V/F, justifique)** → **Falso**: el dinámico se adapta mientras procesa (arranca con el NYT y se reordena con la Sibling Property), el estático se arma con las frecuencias del mensaje completo ya conocidas. Ejemplo del material: símbolos A, C, S → estático `A=1, C=00, S=01`; dinámico `A=0, S=10, C=11`.
6. **Peor canal ternario uniforme** → todas las probabilidades en 1/3, $C = 0$; emisor y receptor se tratan como independientes (el receptor "ya sabe" lo que va a llegar, no hay información nueva).
7. **¿Los métodos estáticos son de doble lectura? ¿Con o sin pérdida?** → Sí, doble lectura (primero cuentan frecuencias, después codifican); son sin pérdida.
8. **¿A mayor o menor redundancia se comprime más?** → A mayor redundancia, mayor margen de compresión (más patrones repetidos para explotar).
9. **Peor BSC** → $p = 0{,}5$, $C = 0$ (nunca se sabe si el bit recibido corresponde a un 0 o a un 1 enviado).
10. **¿Por qué un archivo comprimido tiene mayor entropía y menor redundancia que el original?** → porque la compresión elimina justamente los patrones predecibles; lo que queda es más "sorpresivo" en promedio (más incertidumbre por símbolo).
11. **BWT inversa** → reconstruir la cadena original a partir de la transformada, reordenando columnas iterativamente hasta encontrar la fila terminada en el símbolo de fin de cadena (ejemplo resuelto en 11.7).
12. **Si se aplicara BWT antes de armar la matriz de transición de Markov, ¿cómo serían los resultados de la compresión (Recuperatorio)?** → Mejores. BWT agrupa símbolos iguales, entonces después de cada símbolo es muy probable el mismo símbolo: la matriz de transición queda con valores próximos a 1 en pocas celdas, la entropía condicional baja, y la longitud promedio de Shannon/Huffman baja y se acerca a la entropía de la fuente. (La respuesta del alumno decía solo "sería mucho menor" y el docente puso "¿Por qué?": **hay que justificarlo**.)
13. **Si hay que comprimir y encriptar, ¿en qué orden?** → **Primero comprimir y después encriptar.** La compresión se apoya en las frecuencias/patrones del texto; el cifrado (sobre todo Vigenère) rompe esas frecuencias y sube la entropía, así que un texto ya cifrado casi no se comprime. (Y cifrar *después* de comprimir además esconde la estructura.)
14. **Entre transposición y Vigenère, ¿cuál es más seguro y por qué?** → Vigenère: usa varios alfabetos de sustitución, "rompe bastante las probabilidades de ocurrencia de los símbolos" (aplana la distribución de frecuencias, sube la entropía de 3,605 a 4,015 bits en el ejemplo), y por eso es más difícil de atacar por análisis de frecuencias. La transposición solo reordena las letras: conserva las frecuencias.
15. **¿Cuál es el peor y el mejor canal ternario?** → Peor: toda la matriz en $1/3$, $C = 0$ (emisor y receptor independientes). Mejor: la identidad (1 en la diagonal), $C = \log_2 3 = 1{,}585$ bits.
16. **Dada una fuente binaria, ¿cuál es el peor canal BSC?** → $\begin{bmatrix} 0{,}5 & 0{,}5 \\ 0{,}5 & 0{,}5 \end{bmatrix}$: la salida no depende de la entrada, nunca se sabe si un 1 recibido fue un 0 o un 1 enviado; $C = 0$.
17. **¿Los archivos comprimidos tienen más o menos entropía/redundancia?** → ver #10 (mayor entropía por símbolo, redundancia casi nula).
18. **Propiedades de la cantidad de información** I → ver sección 1.1 (positiva, inversa a p, crece con la cantidad de sucesos, aditiva, H máxima con equiprobabilidad).
19. **¿En qué se basan y en qué difieren PCM y DPCM?** → PCM cuantifica y codifica **cada muestra**; DPCM codifica la **diferencia** entre muestras sucesivas (rango menor ⇒ menos bits por muestra para el mismo error, menos ancho de banda, más tolerancia al ruido). Cuenta de ejemplo en 3.5 y 3.7.
20. **¿Cuándo se pasa a Huffman dinámico y cuál es el objetivo de la Sibling Property?** → Cuando no se conocen las frecuencias de antemano (compresión en una sola lectura). La Sibling Property (pesos no decrecientes numerando de abajo hacia arriba y de izquierda a derecha) se aplica cuando al incrementar un peso se rompe ese orden — pasos 3, 5, 7 y 9 de `ABRACADABRA`, los cuatro "ACORDATE QUE ACÁ TAMBIÉN SE HACE EL BALANCEO" del Excalidraw — y su objetivo es **que el árbol siga siendo un árbol de Huffman óptimo tras cada actualización**. Fundamento: un árbol binario es de Huffman **si y solo si** cumple la sibling property (Gallager), así que conservarla equivale a tener siempre un código óptimo para las frecuencias vistas, sin reconstruir el árbol. Procedimiento, numeración y ejemplo del Tema 2 en 6.7.1.

---

10. Recetario: cómo se hace cada método, paso a paso (y dónde se cae la gente)

Formato: *qué hacer → qué entregar → trampa conocida*. El ejemplo numérico de la mayoría está en la sección 6 (`ABRACADABRA`) o en la 11 (formato de parcial).

10.1 Entropía, redundancia y comparación con código fijo

1. Contar frecuencias → $p_i = f_i/n$. 2. $H = \sum p_i \log_2(1/p_i)$. 3. $H_{max} = \log_2 q \to R = 1 - H/H_{max}$. 4. Código fijo: $L_{fijo} = \lceil \log_2 q \rceil$; bits totales $= n \cdot L_{fijo}$; compresión $= 1 - L/L_{fijo}$.
   Trampa: no mezclar las dos "redundancias" (ver 1.3); acordate de decir qué fórmula usás. Con base $r \neq 2$ dividir por $\log_r q$.

10.2 Shannon

1. Ordenar p_i de mayor a menor. 2. $l_i = \lceil \log_2(1/p_i) \rceil$ (**entero exacto ⇒ ese entero**). 3. $FA_i = \sum_{j<i} p_j$. 4. FA_i a binario (×2 sucesivos), quedarse con los primeros l_i dígitos. 5. $L = \sum p_i l_i$, comparar con el fijo.
   Trampa: sumar 1 cuando $\log_2(1/p)$ es entero (error marcado por el docente).

10.3 Huffman estático

1. Ordenar. 2. Sumar los dos menores, reinsertar la suma en su lugar, repetir hasta 1. 3. Etiquetar 0/1 en cada rama y leer raíz→hoja. 4. L y $\eta = H/L$.
   Trampas: hay varios árboles válidos (todos con el mismo L); si el enunciado pide "Huffman base r", se combinan r símbolos por paso; verificar Kraft $= 1$ y que ningún código sea prefijo de otro.

10.4 Fano

1. Ordenar de mayor a menor. 2. Cortar la lista donde las dos sumas queden lo más cerca posible de 0,5. 3. `0` al grupo de arriba y `1` al de abajo. 4. Repetir en cada grupo hasta quedar con 1 símbolo.
   Trampas: si hay dos cortes igual de buenos, cualquiera sirve (probar ambos); **no es Huffman** (Fano arma el árbol de arriba hacia abajo y a veces da más largo que Huffman).

10.5 Codificación aritmética

1. Intervalo $[Q_s, R_s)$ de cada símbolo = acumulada de los anteriores → acumulada + p_s (siempre en el **mismo orden**). 2. Arrancar en $[0, 1)$. 3. Por cada símbolo: $rango = high - low$; $low' = low + rango \cdot Q_s$; $high' = low + rango \cdot R_s$. 4. La salida es **cualquier número del intervalo final**; el de menos dígitos es el mejor. 5. Bits $\approx -\log_2 \prod p_i = -\log_2(high - low)$.

- **"Con solo 3 decimales … hasta que no logren diferenciarse las cotas"**: redondear a 3 decimales **en cada paso**; parar al llegar al símbolo en que $low = high$ (ese símbolo ya no se puede codificar). Se codifica hasta el anterior (11.5).
  Trampa: usar una tabla de intervalos de un orden y recorrer otro; acordarse de redondear cada cota, no solo la última.

10.6 Lempel-Ziv con ventana deslizante (LZ77)

1. Dos buffers: de **búsqueda** (lo ya codificado, acotado por el tamaño de ventana) y de **mirar más allá** (lo que falta). 2. En cada paso buscar en el de búsqueda la **coincidencia más larga** con el comienzo de lo que falta. 3. Emitir `(desplazamiento, longitud, siguiente símbolo)` ; sin coincidencia: `(0,0,símbolo)` . 4. Avanzar $longitud + 1$ símbolos.
   Trampas: con "ventana de 8 caracteres" el desplazamiento máximo es 8 (los caracteres más viejos "salen"); contar el desplazamiento **hacia atrás** desde la posición actual; al comparar con "código de igual longitud" contar los bits de cada campo según el enunciado.

10.7 Lempel-Ziv-Welch (LZW)

1. Diccionario inicial con el alfabeto (índices $0 \ldots q-1$ o ASCII). 2. Leer símbolos mientras la cadena acumulada I + próximo símbolo **esté** en el diccionario. 3. Cuando $I + x$ no está: **emitir el índice de** I, agregar $I + x$ con el siguiente índice libre y reiniciar $I = x$. 4. Al final emitir el índice de I.
   Trampas: no emitir nada en los pasos donde "sigue acumulando"; el ancho del código (bits) depende del tamaño final del diccionario. Orden recomendado con Huffman: **LZW primero, Huffman después**.

10.8 Huffman dinámico

Receta completa y ejemplo de 35 bits en 6.7; decodificación en 11.8.
Trampas: el NYT va **a la izquierda** (`0`) y la hoja nueva a la derecha; el símbolo nuevo se emite como código del NYT **+ código fijo**; después de cada símbolo revisar la Sibling Property de abajo hacia arriba; los empates los fija la convención del enunciado. Para decidir si hay swap: numerá los nodos de abajo hacia arriba y de izquierda a derecha, buscá el último nodo con el mismo peso del que vas a incrementar (si no es él ni su padre, swap) y verificá al final que la lista de pesos no baje (6.7.1).

10.9 BWT (directa e inversa)

- **Directa**: agregar `$` → todas las rotaciones → ordenar → última columna. (`$` es menor que cualquier letra.)

- **Inversa**: escribir la última columna L; ordenarla ⇒ primera columna F; **pegar L a la izquierda de la tabla, reordenar y repetir n veces**; la fila que termina en `$` es el original (11.7).
- Para qué sirve: agrupa símbolos iguales ⇒ después comprime mejor RLE/Huffman/Markov.

10.10 Fuente de Markov de orden 1

Contar pares (cierre circular) → dividir cada fila por las apariciones del origen → π resolviendo $\pi P = \pi$ → ergodicidad con el grafo → $H(S_M) = \sum \pi_i H(\text{fila}_i)$ → $p(\text{cadena})$ → código por estado (Huffman/Fano/Shannon) → $L_M = \sum \pi_i L_i$. Ejemplos completos: 11.2 y 11.3.

10.11 Canal

Matriz → $P(b_j) = \sum_i P(a_i)P(b_j \mid a_i)$ → $H(A)$, $H(B)$, $H(B \mid A) = \sum_i P(a_i) H(\text{fila}_i)$ → $I = H(B) - H(B \mid A)$ (verificar $0 \le I \le \min(H(A), H(B))$ y $I \le C$). Capacidad: BSC $1 - H(p)$; simétrico $\log_2 q - H(\text{fila})$; asimétrico: maximizar sobre $P(a)$. Ejemplo: 11.4.

10.12 Codificación predictiva (K óptimo)

Rachas de ceros antes de cada `1` (+ ceros finales) → contenedores de K bits, con $2^K - 1$ = "continúa" → probar $K = 2, 3, 4$ → quedarse con el de menos bits y verificar que $K + 1$ empeora (11.6).

10.13 Discretización PCM/DPCM

Rango (de la señal en PCM, de las diferencias en DPCM) ÷ ancho del segmento ⇒ niveles ⇒ $\lceil \log_2 \rceil$. Con valor medio: **ancho** $= 2\times$ **error** (3.5).

10.14 Cifrado (transposición + Vigenère)

Cifrar: transponer cada bloque con la clave de permutación y después sumar la clave de Vigenère letra a letra mod 26. Descifrar **en orden inverso**: restar Vigenère, después deshacer la permutación (11.9).

10.15 Cómo comparar métodos (la pregunta "comparar los resultados")

Siempre con **bits totales** y **bits/símbolo** sobre la **misma** cadena, más la referencia del código de bloque y la cota $n \cdot H$. Escribir las dos conclusiones esperables: *Huffman ≤ Fano ≤ Shannon en L* y *los métodos adaptativos/de diccionario pagan costo de aprendizaje en mensajes cortos*.

---

11. Ejercicios tipo del parcial, resueltos paso a paso

Todos los números de esta sección fueron recalculados por computadora. Las fuentes son las de los enunciados reales (vocales Tema 1 y 3, parcial 2022, parcial 10-10-2023, Recuperatorio, Condicional).

11.1 Bloque A del Tema 1: fuente independiente (entropía, redundancia, Shannon, comparación)

Enunciado: `…aaaae|aeieouaaiiuaoaueeooiiuuaoaaaae|aei…` ; entre barras: `a e i e o u a a i i u a o a u e e o o i i u u a o a a a a e` ($n = 30$, $q = 5$). Hallar entropía, redundancia, codificar por Shannon y comparar con un código de igual longitud.

1. **Frecuencias**: $a = 10$, $e = i = o = u = 5$ ⇒ $p_a = 1/3$, $p_e = p_i = p_o = p_u = 1/6$.
2. **Entropía**: $H = \frac{1}{3}\log_2 3 + 4 \cdot \frac{1}{6}\log_2 6 = \mathbf{2{,}2516}$ bits/símbolo.
3. **Redundancia**: $H_{max} = \log_2 5 = 2{,}3219$; $\eta = 96{,}97\%$; $R = \mathbf{3{,}03\%}$.
4. **Shannon** (orden a, e, i, o, u):

s_i	p_i	FA	FA **en binario**	$\log_2(1/p_i)$	l_i	**Código**
a	10/30	0	0,0000…	1,585	2	`00`
e	5/30	0,3333	0,0101…	2,585	3	`010`
i	5/30	0,5000	0,1000…	2,585	3	`100`
o	5/30	0,6667	0,1010…	2,585	3	`101`
u	5/30	0,8333	0,1101…	2,585	3	`110`

5. **Longitud media**: $L = \frac{1}{3} \cdot 2 + 4 \cdot \frac{1}{6} \cdot 3 = 2{,}6667$ bits/símbolo ⇒ $30 \cdot 2{,}6667 = \mathbf{80}$ bits.
6. **Comparación**: código fijo $\lceil \log_2 5 \rceil = 3$ bits ⇒ 90 bits. Compresión $= 1 - 80/90 = \mathbf{11{,}11\%}$. Eficiencia de Shannon $\eta = H/L = 2{,}2516/2{,}6667 = 84{,}4\%$.
7. *(Para comparar)* Huffman sobre la misma fuente: $L = 2{,}3333$ ($\eta = 96{,}5\%$), o sea 70 bits; Shannon se queda más lejos porque redondea hacia arriba cada símbolo.

11.2 Bloque B del Tema 1: Markov de orden 1 — matriz, estacionaria, ergodicidad, entropía, $p(eeii)$ y codificación por estado

Paso 1 — contar pares consecutivos (cierre circular: el último `e` va al primer `a` del bloque siguiente):

origen \ siguiente	a	e	i	o	u	Σ
a	4	2	1	2	1	10
e	1	1	1	2	0	5
i	0	1	2	0	2	5
o	2	0	1	1	1	5
u	3	1	0	0	1	5

Paso 2 — matriz de transición (dividir cada fila por su suma; cada fila suma 1):

$$P = \begin{bmatrix} 0{,}4 & 0{,}2 & 0{,}1 & 0{,}2 & 0{,}1 \\ 0{,}2 & 0{,}2 & 0{,}2 & 0{,}4 & 0 \\ 0 & 0{,}2 & 0{,}4 & 0 & 0{,}4 \\ 0{,}4 & 0 & 0{,}2 & 0{,}2 & 0{,}2 \\ 0{,}6 & 0{,}2 & 0 & 0 & 0{,}2 \end{bmatrix} \qquad (\text{orden } a, e, i, o, u)$$

Paso 3 — estacionaria, elevando P a potencias altas. En vez de resolver el sistema $\pi P = \pi$, se calcula $P^2, P^4, P^8, P^{16} \ldots$ (cada paso es elevar al cuadrado el anterior) hasta que **todas las filas se vuelvan iguales**; esa fila es π y la matriz resultante es la **matriz estacionaria**.

$$P^2 = \begin{bmatrix} 0{,}34 & 0{,}16 & 0{,}16 & 0{,}20 & 0{,}14 \\ 0{,}28 & 0{,}12 & 0{,}22 & 0{,}20 & 0{,}18 \\ 0{,}28 & 0{,}20 & 0{,}20 & 0{,}08 & 0{,}24 \\ 0{,}36 & 0{,}16 & 0{,}16 & 0{,}12 & 0{,}20 \\ 0{,}40 & 0{,}20 & 0{,}10 & 0{,}20 & 0{,}10 \end{bmatrix}$$

$$P^4 = \begin{bmatrix} 0{,}333 & 0{,}166 & 0{,}168 & 0{,}165 & 0{,}169 \\ 0{,}334 & 0{,}171 & 0{,}165 & 0{,}158 & 0{,}172 \\ 0{,}332 & 0{,}170 & 0{,}166 & 0{,}170 & 0{,}163 \\ 0{,}335 & 0{,}168 & 0{,}164 & 0{,}171 & 0{,}162 \\ 0{,}332 & 0{,}160 & 0{,}170 & 0{,}172 & 0{,}166 \end{bmatrix}$$

$$P^8 \approx \begin{bmatrix} 0{,}3333 & 0{,}1667 & 0{,}1667 & 0{,}1667 & 0{,}1667 \\ 0{,}3333 & 0{,}1667 & 0{,}1667 & 0{,}1667 & 0{,}1667 \\ 0{,}3333 & 0{,}1667 & 0{,}1667 & 0{,}1667 & 0{,}1667 \\ 0{,}3334 & 0{,}1667 & 0{,}1666 & 0{,}1666 & 0{,}1667 \\ 0{,}3333 & 0{,}1667 & 0{,}1667 & 0{,}1668 & 0{,}1666 \end{bmatrix}$$

$$P^{16} = \begin{bmatrix} 0{,}3333 & 0{,}1667 & 0{,}1667 & 0{,}1667 & 0{,}1667 \\ 0{,}3333 & 0{,}1667 & 0{,}1667 & 0{,}1667 & 0{,}1667 \\ 0{,}3333 & 0{,}1667 & 0{,}1667 & 0{,}1667 & 0{,}1667 \\ 0{,}3333 & 0{,}1667 & 0{,}1667 & 0{,}1667 & 0{,}1667 \\ 0{,}3333 & 0{,}1667 & 0{,}1667 & 0{,}1667 & 0{,}1667 \end{bmatrix}$$

En P^2 las columnas todavía son distintas; en P^4 ya se parecen (±0,01); en P^8 coinciden a 3 decimales y en P^{16} las 5 filas son idénticas. Por lo tanto

$$\pi = \left(\tfrac{1}{3},\ \tfrac{1}{6},\ \tfrac{1}{6},\ \tfrac{1}{6},\ \tfrac{1}{6}\right)$$

Chequeo rápido: π coincide con las frecuencias de la secuencia ($10/30$ y $5/30$) y cumple $\pi P = \pi$. *(Que las filas lleguen a ser iguales además confirma la ergodicidad: el estado inicial deja de importar.)*

Paso 4 — ¿ergódico? Grafo: hay un ciclo que pasa por todos los estados, $a \to e \to o \to i \to u \to a$ $(0{,}2;\ 0{,}4;\ 0{,}2;\ 0{,}4;\ 0{,}6)$, así que desde cualquiera se llega a cualquiera (irreducible); además $a \to a$ es un bucle (aperiódica), y P^2 ya tiene todos los elementos positivos. **Sí es ergódico**, a pesar de los ceros de la matriz (ver advertencia en 5.2). Si en el examen se espera "no ergódico porque tiene ceros", justificá con el criterio de accesibilidad igualmente.

Paso 5 — entropía de la fuente (por fila y ponderada por π):
$H(a) = 2{,}1219$; $H(e) = 1{,}9219$; $H(i) = 1{,}5219$; $H(o) = 1{,}9219$; $H(u) = 1{,}3710$.

$$H(S_M) = \tfrac{1}{3} \cdot 2{,}1219 + \tfrac{1}{6}(1{,}9219 + 1{,}5219 + 1{,}9219 + 1{,}3710) = \mathbf{1{,}830}\ \text{bits/símbolo}\ \ (\leq H(S) = 2{,}2516)$$

Paso 6 — probabilidad de una cadena: $p(eeii) = \pi_e \cdot P(e|e) \cdot P(i|e) \cdot P(i|i) = \tfrac{1}{6} \cdot 0{,}2 \cdot 0{,}2 \cdot 0{,}4 = \tfrac{1}{375} = \mathbf{0{,}002667}$.

Paso 7 — codificar "como fuente de Markov de orden 1" con Huffman, Fano y Shannon. Se arma un código **por estado** (por fila), usando solo los sucesores con probabilidad > 0 de esa fila:

Estado	Sucesores y p	Huffman	Fano	Shannon
a	a .4, e .2, o .2, i .1, u .1	a `11` e `00` o `01` i `100` u `101` ($L = 2{,}2$)	a `0` e `10` o `110` i `1110` u `1111` ($2{,}2$)	a `00` e `011` o `100` i `1100` u `1110` ($2{,}8$)
e	o .4, a .2, e .2, i .2	a `00` e `01` i `10` o `11` ($2{,}0$)	o `0` a `10` e `110` i `111` ($2{,}0$)	o `00` a `011` e `100` i `110` ($2{,}6$)
i	i .4, u .4, e .2	u `0` e `10` i `11` ($1{,}6$)	i `0` u `10` e `11` ($1{,}6$)	i `00` u `01` e `110` ($2{,}2$)
o	a .4, i .2, o .2, u .2	i `00` o `01` u `10` a `11` ($2{,}0$)	a `0` i `10` o `110` u `111` ($2{,}0$)	a `00` i `011` o `100` u `110` ($2{,}6$)
u	a .6, e .2, u .2	a `1` e `00` u `01` ($1{,}4$)	a `0` e `10` u `11` ($1{,}4$)	a `0` e `100` u `110` ($1{,}8$)

(Las asignaciones de Huffman con empates no son únicas: cualquiera con las mismas longitudes es válida.)

Longitud media de Markov $L_M = \sum \pi_i L_i$:

- Huffman: $\frac{1}{3} \cdot 2{,}2 + \frac{1}{6}(2{,}0 + 1{,}6 + 2{,}0 + 1{,}4) = \mathbf{1{,}900}$ bits/símbolo.
- Fano: $\mathbf{1{,}900}$ (empata con Huffman).
- Shannon: $\frac{1}{3} \cdot 2{,}8 + \frac{1}{6}(2{,}6 + 2{,}2 + 2{,}6 + 1{,}8) = \mathbf{2{,}467}$.

Método	L **(bits/sím.)**	**Eficiencia**
Código de bloque	3,000	—
Shannon, fuente independiente	2,667	$H/L = 84{,}4\%$
Huffman, fuente independiente	2,333	96,5 %
Shannon, Markov	2,467	$H(S_M)/L_M = 74{,}2\%$
Huffman / Fano, Markov	**1,900**	$H(S_M)/L_M = 96{,}3\%$
Cota (entropía de Markov)	*1,830*	*100 %*

Ejemplo de mensaje: `a e i e o`. Primer símbolo con bloque fijo ($a = 000, e = 001, i = 010, o = 011, u = 100$): `000`; luego $a \to e$ con el código del estado a: `00`; $e \to i$ con el del estado e: `10`; $i \to e$ con el del estado i: `10`; $e \to o$ con el del estado e: `11`. Total `000 00 10 10 11` $= 11$ bits contra 15 del código de bloque. Decodificar es el camino inverso: el estado anterior dice qué tabla usar para leer el siguiente código.

Conclusiones a escribir: Markov mejora porque conocer el estado previo reduce la incertidumbre ($H(S_M) = 1{,}83 < H(S) = 2{,}25$); Huffman = Fano ≤ Shannon; la desventaja es que hay que conocer/transmitir las 5 tablas.

11.3 Markov de orden 1 sobre `ABRACADABRA` (el mismo ejemplo del integrador, versión corta)

Acá se cuenta la palabra **sin cierre circular** (es un mensaje único, no una trama periódica). Pares: AB, BR, RA, AC, CA, AD, DA, AB, BR, RA (10).

origen \ siguiente	A	B	R	C	D
A (4 veces)	0	2/4	0	1/4	1/4
B (2)	0	0	1	0	0
R (2)	1	0	0	0	0
C (1)	1	0	0	0	0
D (1)	1	0	0	0	0

- **Estacionaria**: $\pi_A = \pi_R + \pi_C + \pi_D$, $\pi_B = \pi_R = \frac{1}{2}\pi_A$, $\pi_C = \pi_D = \frac{1}{4}\pi_A$ ⇒ $\pi = (0{,}4;\ 0{,}2;\ 0{,}2;\ 0{,}1;\ 0{,}1)$ para (A, B, R, C, D).
- **Ergódico**: sí (ciclos $A \to B \to R \to A$ de largo 3 y $A \to C \to A$ de largo 2 ⇒ irreducible y aperiódica), aunque la matriz tiene muchos ceros.
- **Entropía**: solo la fila A tiene incertidumbre: $H(A) = \frac{1}{2}\log_2 2 + 2 \cdot \frac{1}{4}\log_2 4 = 1{,}5$ ⇒ $H(S_M) = 0{,}4 \cdot 1{,}5 = \mathbf{0{,}6}$ bits/símbolo (contra $H(S) = 2{,}04$).
- **Codificación** (Huffman = Fano = Shannon, porque $\frac{1}{2}, \frac{1}{4}, \frac{1}{4}$ son potencias de 2): estado A: B `0`, C `10`, D `11` ($L_A = 1{,}5$). Los estados B, R, C, D tienen **un solo sucesor ($p = 1$) ⇒ $l = \lceil \log_2 1 \rceil = 0$ bits**. $L_M = 0{,}4 \cdot 1{,}5 = \mathbf{0{,}6} = H(S_M)$ ⇒ código **compacto** ($\eta = 100\%$).
- **Mensaje** `ABRACADABRA`: primer símbolo `A` con bloque de 3 bits (`000`); luego $A \to B$: `0` (1 bit); $B \to R$: 0 bits; $R \to A$: 0; $A \to C$: `10` (2 bits); $C \to A$: 0; $A \to D$: `11` (2); $D \to A$: 0; $A \to B$: `0` (1); $B \to R$: 0; $R \to A$: 0. **Total:** $3 + 1 + 2 + 2 + 1 = 9$ **bits** (contra 23 de Huffman independiente y 33 de bloque).

- $p(ABRA) = \pi_A \cdot P(B|A) \cdot P(R|B) \cdot P(A|R) = 0{,}4 \cdot 0{,}5 \cdot 1 \cdot 1 = \mathbf{0{,}2}$.

> ⚠️ Estos 9 bits **no incluyen** las tablas de los estados (ni la marca de fin); es la ganancia del modelo de Markov en una fuente fuertemente dependiente. Con cierre circular la fila A cambiaría (aparecería $A \to A$ con $1/5$): fijate qué convención usa el enunciado.

11.4 Canal: información mutua con todas las entropías (Tema 3 y 4 del examen modelo)

*(Cómo se calcula cada una de las cuatro entropías, con ejemplo ternario y errores típicos: **2.4.1**.)*

Enunciado (Tema 3): BSC con $p(0|0) = 0{,}85$ y $p(0) = 0{,}7$. Hallar la información mutua. *(Tema 4: igual pero $p(1) = 0{,}7$.)*

1. **Matriz**: $\begin{bmatrix} 0{,}85 & 0{,}15 \\ 0{,}15 & 0{,}85 \end{bmatrix}$; entrada $P(A) = (0{,}7;\ 0{,}3)$.
2. **Salida**: $P(b_0) = 0{,}7 \cdot 0{,}85 + 0{,}3 \cdot 0{,}15 = 0{,}64$; $P(b_1) = 0{,}7 \cdot 0{,}15 + 0{,}3 \cdot 0{,}85 = 0{,}36$.
3. **Entropías**: $H(A) = H(0{,}7) = 0{,}8813$; $H(B) = H(0{,}64) = 0{,}9427$; $H(B|A) = 0{,}7 \cdot H(0{,}15) + 0{,}3 \cdot H(0{,}15) = H(0{,}15) = 0{,}6098$ (ruido; en un BSC no depende de $P(A)$).
4. **Información mutua**: $I(A;B) = H(B) - H(B|A) = 0{,}9427 - 0{,}6098 = \mathbf{0{,}333}$ bits.
5. **Verificación por otra vía** (Bayes): $P(a_0|b_0) = \frac{0{,}595}{0{,}64} = 0{,}9297$; $P(a_1|b_0) = 0{,}0703$; $P(a_0|b_1) = \frac{0{,}105}{0{,}36} = 0{,}2917$; $P(a_1|b_1) = 0{,}7083$. Equivocación $H(A|B) = 0{,}64\, H(0{,}9297) + 0{,}36\, H(0{,}2917) = 0{,}5484 \Rightarrow I = H(A) - H(A|B) = 0{,}8813 - 0{,}5484 = 0{,}333$ ✓. Entropía conjunta: $H(A,B) = H(A) + H(B|A) = 1{,}4911$ ✓ $(= H(B) + H(A|B))$.
6. **Para el Tema 4** ($P(1) = 0{,}7$): sale $P(B) = (0{,}36;\ 0{,}64)$ y el mismo $I = 0{,}333$ (el BSC es simétrico, intercambiar 0↔1 no cambia nada).
7. **Capacidad de ese canal**: $C = 1 - H(0{,}15) = 0{,}3902$ bits, con entrada equiprobable. Como debe ser, $I = 0{,}333 \le C = 0{,}390$.

Preguntas relacionadas que suelen venir pegadas: *Capacidad del BSC con $p(0|0) = 0{,}95$* ⇒ $C = 1 - H(0{,}05) = \mathbf{0{,}7136}$. *Con $p(0|0) = 0{,}5$* ⇒ $C = 0$. *Con $p(0|0) = 0{,}85$* ⇒ $C = 0{,}3902$. *BSC $7/8$ del recuperatorio* ⇒ $I = C = 0{,}4564$. *Parcial de Arce $[0{,}8; 0{,}2]$ con $P(a_1) = 0{,}7$* ⇒ $P(B) = (0{,}62; 0{,}38)$, $H(B) = 0{,}958$, $H(B|A) = 0{,}7219$, $I = \mathbf{0{,}2361}$; su capacidad (si piden C) es $1 - 0{,}7219 = \mathbf{0{,}2781}$ (la hoja del alumno confundió $H(B|A)$ con C).

11.5 Aritmética con 3 decimales (Tema 3)

Enunciado: `…aaaae|aeieouaaooeuuuuueooiiuuaoaaaae|aei…` ; entre barras: `aeieouaaooeuuuuueooiiuuaoaaaae` ($n = 30$). Con solo tres decimales, codificar aritméticamente hasta que no se diferencien las cotas.

1. **Frecuencias**: $a = 8$, $u = 8$, $o = 6$, $e = 5$, $i = 3$ ⇒ $p = 0{,}267;\ 0{,}267;\ 0{,}200;\ 0{,}167;\ 0{,}100$.
2. **Intervalos** $[Q, R)$ (orden por probabilidad decreciente; las acumuladas se redondean a 3 decimales):

Símbolo	p	$[Q, R)$
a	8/30	[0,000 ; 0,267)
u	8/30	[0,267 ; 0,533)
o	6/30	[0,533 ; 0,733)
e	5/30	[0,733 ; 0,900)
i	3/30	[0,900 ; 1,000)

3. **Pasos** ($rango = high - low$; cada cota se **redondea a 3 decimales**):

Paso	Símbolo	Cálculo	Intervalo
1	a	$0 + 1 \cdot [0;\ 0{,}267)$	[0,000 ; 0,267)
2	e	$0 + 0{,}267 \cdot 0{,}733 = 0{,}1957;\ 0 + 0{,}267 \cdot 0{,}900 = 0{,}2403$	[0,196 ; 0,240)
3	i	$0{,}196 + 0{,}044 \cdot 0{,}900 = 0{,}2356;\ 0{,}196 + 0{,}044 \cdot 1 = 0{,}240$	[0,236 ; 0,240)
4	e	$0{,}236 + 0{,}004 \cdot 0{,}733 = 0{,}2389;\ 0{,}236 + 0{,}004 \cdot 0{,}900 = 0{,}2396$	[0,239 ; 0,240)
5	o	$0{,}239 + 0{,}001 \cdot 0{,}533 = 0{,}2395;\ 0{,}239 + 0{,}001 \cdot 0{,}733 = 0{,}2397$	[0,240 ; 0,240) ✗ **cotas iguales**

4. **Resultado**: en el 5.º símbolo (o) las cotas coinciden con 3 decimales: **no se puede discernir un valor entre ellas** ⇒ se detiene. Se codificaron **4 símbolos** (`aeie`) y la salida puede ser **0,239** (cualquier número en $[0{,}239;\ 0{,}240)$). *(Truncando en vez de redondear los extremos cambian los últimos decimales — [0,195;0,240), [0,235;0,240), [0,238;0,239), [0,238;0,238) — pero el paso de corte es el mismo: el 5.º. Usá la convención que fije el docente.)*
5. **Comparación**: código de igual longitud $\lceil \log_2 5 \rceil = 3$ bits ⇒ $4 \cdot 3 = 12$ bits. Aritmético: ancho real del intervalo $= \prod p = \frac{8}{30} \cdot \frac{5}{30} \cdot \frac{3}{30} \cdot \frac{5}{30} = 7{,}4 \cdot 10^{-4}$ ⇒ $-\log_2 = 10{,}4 \Rightarrow \mathbf{11}$ bits ⇒ compresión $\approx 8\%$ (en la cadena de 4 símbolos).

*(Fórmulas del método y la versión de precisión completa: 6.5. Ejemplo del Parcial de Nanni Rebollo con $c = [0; 0{,}25)$, $b = [0{,}25; 0{,}5)$, $a = [0{,}5; 1)$: `aabb` ⇒ [0,828125 ; 0,84375), número `0,828125` $= 0{,}110101_2$ ⇒ **6 bits**, los mismos que Huffman (`1 1 00 00`) en esos 4*

símbolos.)

11.6 Codificación predictiva: K óptimo (parcial 10-10-2023, versiones A y B)

Enunciado: la cadena de la compuerta OREX es `10000001010000100000` (20 bits). Hallar el K de mejor compresión.

1. **Rachas de ceros antes de cada `1`**: `1` (0 ceros antes), `0000001` (6), `01` (1), `00001` (4); **ceros finales**: 5.
2. **Para cada K** (el código `1…1` de K bits = "continúa la racha"):

K	**Contenedores**	**Bits**
2 (máx. 3)	0→`00`; 6→`11 11 00`; 1→`01`; 4→`11 01`; 5→`11 10` ⇒ 9 contenedores	**18**
3 (máx. 7)	0→`000`; 6→`110`; 1→`001`; 4→`100`; 5→`101` ⇒ 5 contenedores	**15**
4 (máx. 15)	`0000 0110 0001 0100 0101` ⇒ 5 contenedores	**20**
5	5 contenedores	25

3. **Conclusión**: $K_{opt} = \mathbf{3}$ (15 bits, contra 20 de la cadena original ⇒ 25 % de ahorro). Se corta la búsqueda en $K = 4$: empeora (20 > 15), y más grande empeora todavía.
4. **Correcciones del docente**: no agregar un `1` imaginario al principio (la cadena **ya empieza con** `1`: la primera racha es de 0 ceros, `000`); y **verificar siempre** $K + 1$ antes de afirmar el óptimo.

11.7 BWT inversa (Parcial 2022, Tema I): `amnr$aaag`

Enunciado: dada la secuencia `amnr$aaag`, reordenada con Burrows-Wheeler, hallar la cadena original. Última columna L = `a m n r $ a a a g` ($n = 9$).

Método: en cada iteración se **pega L a la izquierda** de la tabla y se **ordena**; después de $n = 9$ iteraciones la tabla es la matriz completa de rotaciones ordenadas.

Iteración	**Tabla ordenada (primeras filas…)**
1 (solo L ordenada)	`$ a a a a g m n r`
2	`$a a$ ag am an gr ma na ra`
3	`$an a$a agr ama ana gra ma$ nag ram`
4	`$ana a$an agra ama$ anag gram ma$a nagr rama`
…	…
9	`$anagrama a$anagram agrama$an ama$anagr` **`anagrama$`** `grama$ana ma$anagra nagrama$a rama$anag`

La fila que **termina en `$`** es `anagrama$` ⇒ **la cadena original es `anagrama`**.
Verificación: rotaciones de `anagrama$` ordenadas, última columna = `amnr$aaag` ✓.

11.8 Huffman dinámico: decodificación (Recuperatorio, ejercicio 3)

Enunciado: descomprimir `000010010010011010`, sabiendo que el código fijo es A=`00`, N=`01`, D=`10`, B=`11`.

Método: el receptor repite en espejo lo que hace el emisor: bajar por el árbol bit a bit hasta una hoja; si es el **NYT**, leer 2 bits de código fijo y crear la hoja; después actualizar pesos y aplicar la Sibling Property.

Paso	**Bits leídos**	**Interpretación**	**Símbolo**	**Árbol después (código vigente)**
1	`00`	árbol vacío ⇒ símbolo nuevo con fijo `00`	**A**	A=`1`, NYT=`0`
2	`0 01`	NYT=`0` + fijo `01`	**N**	A=`1`, N=`01`, NYT=`00`
3	`00 10`	NYT=`00` + fijo `10`; **swap** (A sube a la izq.)	**D**	A=`0`, N=`11`, D=`101`, NYT=`100`
4	`0`	A=`0` (ya en el árbol)	**A**	A pesa 2
5	`100 11`	NYT=`100` + fijo `11`; **swap** (N sube)	**B**	A=`0`, N=`10`, D=`111`, B=`1101`, NYT=`1100`
6	`0`	A=`0`	**A**	A pesa 3
7	`10`	N=`10`	**N**	N pesa 2

Total: $2+3+4+1+5+1+2 = 18$ bits ✓ (largo del stream). **Mensaje:** `ANDABAN`. Árbol final (pesos): $B_1, D_1 \to 2$; N_2 y ese nodo $2 \to 4$; A_3 + nodo $4 \to 7$ — la resolución de cátedra lo escribe `B1 D1 N2 2 A3 4 7`. Para controlar: **comprimir `ANDABAN` con las mismas reglas devuelve exactamente `000010010010011010`**. *Sibling Property*: se aplica en los pasos **3 y 5** (swap de A con el nodo que contiene al NYT

al abrirlo; en el paso 5, swap de N con el nodo que contiene al NYT (N sube)); en los pasos 1, 2, 4, 6 y 7 el orden de pesos ya se cumple (criterio y numeración en 6.7.1).

11.9 Encriptación: transposición + Vigenère (parcial de Arce)

Enunciado: texto cifrado `VPOJSPSE TCYZJYUR UWYHPTLE PEIQFCLI`. Clave de transposición $(8,7,6,5,4,3,2,1)$ (invertir cada bloque de 8 letras) y clave de Vigenère **BLUE**. A) Descifrar. B) Entropía del original y del cifrado. C) ¿Cuál es más seguro? D) ¿Comprimir antes o después de encriptar?

A) Descifrado — se deshace el último paso aplicado primero: **(1)** restar BLUE letra a letra mod 26 ($A=0,\dots,Z=25$), con la clave repitiéndose:
$V-B=21-1=20=U$; $P-L=15-11=4=E$; $O-U=14-20=-6\equiv 20=U$; $J-E=9-4=5=F$ ⇒ … ⇒ `UEUFREYA SREVINAN TLEDOIRA OTOMERRE`.
(2) Invertir cada bloque de 8 (clave 8,7,…,1) ⇒ `AYERFUEU NANIVERS ARIODELT ERREMOTO` ⇒ **"AYER FUE UN ANIVERSARIO DEL TERREMOTO"**.
(Cifrar es el camino inverso: transponer los bloques y luego sumar la clave.)

B) Entropías: $H(\text{original})=\mathbf{3{,}605}$ bits/símbolo; $H(\text{cifrado})=\mathbf{4{,}015}$. El cifrado sube la entropía: reparte las frecuencias y oculta la estructura del idioma.
C) Más seguro: **Vigenère**, porque usa varios alfabetos de sustitución y "rompe bastante las probabilidades de ocurrencia de los símbolos"; la transposición conserva las frecuencias de las letras.
D) Orden: **primero comprimir y después encriptar**: la compresión se basa en las frecuencias de aparición; un texto ya cifrado casi no es compresible.

11.10 Parcial 07-10-2025, Tema 1 (vocales): Huffman + Markov + canal + discretización — resuelto y con la resolución de un alumno corregida

Enunciado: `….UAIEA-AEIOUUOIEAOOUUAAEEIIAAOAEOAUAIEA-AEI….` (entre guiones, $n=32$, $q=5$).

- **A) Como fuente independiente**: A1) redundancia; A2) codificar por **Huffman** y comparar el grado de compresión con el código de **menor longitud fija** necesario para ese universo de símbolos.
- **B) Como fuente dependiente de primer orden**: B1) entropía de la fuente, ¿ergódico?, matriz de transición; B2) codificar por Huffman y comparar con el Huffman de la fuente independiente.
- **Canal**: BSC con $P(0/0)=0{,}9$, hallar C; matriz del **peor canal ternario uniforme** y comprobar que su C lo define como peor.
- **Discretización**: señal de -5 V a $+5$ V en **PCM con 10 bits**: error de discretización respecto del valor medio de cada segmento; B) con **ADPCM** y rango dinámico de la diferencia muestra–predicción de 1 V ($-0{,}5$ a $+0{,}5$ V): ¿cuántos bits para mantener el error del PCM?

A1) Entropía y redundancia

Frecuencias: $A=10,\ E=6,\ O=6,\ I=5,\ U=5 \Rightarrow p=(10,6,6,5,5)/32$.
$H=\frac{10}{32}\log_2\frac{32}{10}+2\cdot\frac{6}{32}\log_2\frac{32}{6}+2\cdot\frac{5}{32}\log_2\frac{32}{5}=\mathbf{2{,}2669}$ bits/símbolo. $H_{max}=\log_2 5=2{,}3219 \Rightarrow \eta=97{,}63\%$ y $R=1-H/H_{max}=\mathbf{2{,}37\%}$.
Lectura: fuente casi equiprobable, casi sin redundancia que explotar.

A2) Huffman (independiente) y comparación

Unir los dos menores: $I+U=10$; $E+O=12$; después $A+(I+U)=20$ (con $A=10$ y $I{+}U=10$ los dos menores); raíz $=20+12$. Un código válido: $A=$ `00`, $I=$ `010`, $U=$ `011`, $E=$ `10`, $O=$ `11` (Kraft $=1$, instantáneo).
$L=\frac{10\cdot2+5\cdot3\cdot2+6\cdot2\cdot2}{32}=\frac{74}{32}=\mathbf{2{,}3125}$ bits/símbolo; $\eta=H/L=98{,}0\%$.
Código fijo: $\lceil\log_2 5\rceil=3$ bits ⇒ 96 bits; Huffman: 74 bits ⇒ compresión $=1-74/96=\mathbf{22{,}92\%}$.

B1) Matriz de transición, ergodicidad y entropía (con cierre circular, como en 11.2)

Pares consecutivos (32, incluyendo el último `A` →primer `A` del bloque siguiente):

origen \ siguiente	A	E	I	O	U	Σ
A	3	3	1	2	1	10
E	2	1	2	1	0	6
I	1	2	1	1	0	5
O	2	0	1	1	2	6
U	2	0	0	1	2	5

$$P = \begin{bmatrix} 0{,}3 & 0{,}3 & 0{,}1 & 0{,}2 & 0{,}1 \\ 0{,}333 & 0{,}167 & 0{,}333 & 0{,}167 & 0 \\ 0{,}2 & 0{,}4 & 0{,}2 & 0{,}2 & 0 \\ 0{,}333 & 0 & 0{,}167 & 0{,}167 & 0{,}333 \\ 0{,}4 & 0 & 0 & 0{,}2 & 0{,}4 \end{bmatrix} \quad (A, E, I, O, U)$$

- **Estacionaria**: con el cierre circular π es **exactamente** la frecuencia de cada símbolo: $\pi = (10, 6, 5, 6, 5)/32 = (0{,}3125;\ 0{,}1875;\ 0{,}15625;\ 0{,}1875;\ 0{,}15625)$ para (A, E, I, O, U). Chequeo elevando P: P^{16} ya tiene las 5 filas iguales a ese vector (en P^8 coinciden a 3 decimales).
- **¿Ergódico? Sí.** Irreducible: $E \to A$, $I \to A$, $O \to A$, $U \to A$ y $A \to E, I, O, U$ ⇒ desde cualquier estado se llega a cualquier otro (ej.: $E \to A \to U$, $U \to A \to E$); aperiódico: todos los estados tienen bucle ($A \to A$, $E \to E$, …) y P^2 ya es todo positivo. **Los ceros** ($E \to U$, $I \to U$, $O \to E$, $U \to E$, $U \to I$) no rompen la ergodicidad porque hay caminos de 2 pasos; sí la romperían si algún estado fuera inalcanzable.
- **Entropías por fila**: $H(A) = 2{,}1710$; $H(E) = 1{,}9183$; $H(I) = 1{,}9219$; $H(O) = 1{,}9183$; $H(U) = 1{,}5219$ ⇒ $H(S_M) = \sum \pi_i H_i = \mathbf{1{,}9359}$ bits/símbolo ($\leq H(S) = 2{,}2669$).

B2) Huffman como fuente de Markov y comparación con el independiente

Método 1 (el de 11.2: un código por estado), usando los sucesores con $p > 0$ de cada fila:

Estado	Sucesores y p	Huffman	$\bar{L}_i$
A	A .3, E .3, O .2, I .1, U .1	A `10` E `11` O `00` I `010` U `011`	2,2
E	A .333, I .333, E .167, O .167	A `10` I `11` E `00` O `01`	2,0
I	E .4, A .2, I .2, O .2	E `11` A `00` I `01` O `10`	2,0
O	A .333, U .333, I .167, O .167	A `10` U `11` I `00` O `01`	2,0
U	A .4, U .4, O .2	A `0` U `10` O `11`	1,6

$L_M = \frac{10 \cdot 2{,}2 + 6 \cdot 2 + 5 \cdot 2 + 6 \cdot 2 + 5 \cdot 1{,}6}{32} = \frac{64}{32} = \mathbf{2{,}000}$ bits/símbolo ⇒ $\eta = H(S_M)/L_M = 96{,}8\%$ ($H(S_M) = 1{,}9359 \leq 2{,}000$ ✓).
Comparación: independiente $L = 2{,}3125$ (74 bits) vs. Markov $L_M = 2{,}000$ (64 bits) ⇒ mejora $1 - 2/2{,}3125 = \mathbf{13{,}5\%}$ (y $33{,}3\%$ respecto del código fijo de 96 bits). *(Más el primer símbolo con bloque fijo de 3 bits y sin contar las 5 tablas.)*

Método 2 (extensión de orden 2: Huffman sobre pares): la distribución conjunta de pares es $\pi_i p_{ij} = n_{ij}/32$ (20 pares distintos). Huffman óptimo sobre esos pares:

Pares (frecuencia)	Código
AA (3)	`000`
AE (3)	`1111`
AO, EA, EI, IE, OA, OU, UA, UU (2 c/u)	`0111` , `0110` , `0010` , `0101` , `1001` , `0011` , `1000` , `0100`
AI, AU, EE, EO, IA, II, IO, OI, OO, UO (1 c/u)	`11101` , `11100` , `11000` , `11011` , `11010` , `11001` , `10100` , `10110` , `10111` , `10101`

$L_2 = 135/32 = 4{,}21875$ bits/par ⇒ $\mathbf{2{,}1094}$ bits/símbolo (Kraft $= 1$). Cota: $(H(S) + H(S_M))/2 = 2{,}1014 \leq 2{,}1094$ ✓. Mejora respecto del independiente: $1 - 2{,}1094/2{,}3125 = \mathbf{8{,}8\%}$. Ante la duda sobre qué método pide el enunciado, aclarar cuál usaste; el de por estado es el de 11.2.

Canal

- **BSC** $P(0/0) = 0{,}9$: matriz $\begin{bmatrix} 0{,}9 & 0{,}1 \\ 0{,}1 & 0{,}9 \end{bmatrix}$; con entrada equiprobable $H(B) = 1$ y $H(B/A) = H(0{,}1) = 0{,}1 \log_2 10 + 0{,}9 \log_2 \frac{1}{0{,}9} = 0{,}3322 + 0{,}1368 = 0{,}4690$ ⇒ $C = 1 - 0{,}4690 = \mathbf{0{,}5310}$ bits/uso.
- **Peor canal ternario uniforme**: las 9 probabilidades condicionales valen $1/3$. $H(B/A) = \log_2 3 = 1{,}585$ y la salida es uniforme ($H(B) = 1{,}585$) ⇒ $I = 0$ ⇒ $C = \mathbf{0}$: emisor y receptor independientes.

Discretización (PCM 10 bits y ADPCM)

- **A) PCM**: rango 10 V, $2^{10} = 1024$ segmentos ⇒ $\Delta = 10/1024 = 9{,}766$ mV; con el valor medio de cada segmento, el error es $\pm\Delta/2 = \pm\mathbf{4{,}883}$ mV, **igual en todos los segmentos**. *(Con 1024 segmentos no hay un nivel en 0 V: los segmentos que rodean al 0 son* $[-\Delta, 0]$ *y* $[0, \Delta]$*, con valores medios* $\mp 4{,}883$ *mV.)*
- **B) ADPCM**: para el mismo error hace falta el mismo Δ: niveles $= 1\ \text{V}/9{,}766\ \text{mV} = 102{,}4 \to 103$ ⇒ $\lceil \log_2 103 \rceil = \mathbf{7}$ bits (con 7 bits el error real es $1/128/2 = 3{,}9$ mV $\leq 4{,}88$ mV ✓; con 6 bits sería $7{,}8$ mV ✗). Atajo: el rango baja 10 veces ⇒ $\log_2 10 = 3{,}32$ ⇒ $10 - 3{,}32 = 6{,}68 \to 7$. Se ahorran 3 bits por muestra (30 %).

Qué había que corregir en la resolución del alumno (Moya, Tema 1)

La resolución tiene bien A1 ($2{,}2669$; $R = 2{,}37\%$), A2 (74 bits, $22{,}92\%$), la conclusión de ergodicidad, el BSC ($C = 0{,}531$), el ternario ($C = 0$) y la parte B de la discretización (7 bits). Se corrige:

1. **B2 — el Huffman por bigramas no era un código válido.** El código que armó **no es instantáneo**: `AO` = `01010` es prefijo de `II` = `010100` y de `IA` = `010101`; `AA` = `01011` es prefijo de `EO` = `010110` y de `EE` = `010111`. Además su Kraft da $0{,}969 < 1$ (árbol incompleto ⇒ no es óptimo). La causa: unió los diez pares de peso 1 "en un solo paso" (*"Iteración 1–9… → Nodo1"*); hay que unirlos **de a dos**, reinsertando cada suma en el orden. Su $L = 142/31/2 = 2{,}29$ bits/símbolo y su "mejora del $1{,}02\%$" quedan mal: el Huffman óptimo sobre esos mismos 31 pares da $131/31/2 = 2{,}113$ (mejora $8{,}6\%$), y con el cierre circular de este apunte $2{,}109$ ($8{,}8\%$); con un código por estado, $2{,}000$ ($13{,}5\%$).
2. **B1 — mezcla de convenciones.** Armó la matriz **sin** cierre circular (31 pares; fila A: $2/9, 3/9, 1/9, 2/9, 1/9$) pero ponderó con $\pi =$ frecuencias, que **solo es la estacionaria si se cierra el círculo**. Con matriz lineal la estacionaria es $(0{,}290; 0{,}194; 0{,}161; 0{,}194; 0{,}161)$. Elegí una convención y mantenela (este apunte: circular). Y $H(E) = H(O) = 1{,}9183$, no $1{,}920$ (su $H_t = 1{,}944$ queda en $1{,}936$).
3. **Discretización — segmento 512.** No tiene valor medio 0 V (ver nota de arriba); el $\pm 4{,}883$ mV es correcto igual.
4. **Detalle de forma**: en el canal ternario escribió $H(Y|X) = -\sum P(X{=}i)\, H(Y|X{=}i)$ con un signo menos de más; es $+\sum$ (cada $H(Y|X{=}i) \geq 0$).
5. **A2/B2**: no está de más escribir la eficiencia $\eta = H/L$ y responder explícitamente "¿mejora?" con el porcentaje.

11.11 Recuperatorio 28-10-2025, Tema 1: Huffman dinámico, BWT + Huffman estático vs. Markov, K óptimo y canal ternario

Enunciado: (1) comprimir `CANTABALA` con Huffman dinámico (código fijo A= `000`, B= `001`, C= `010`, L= `011`, N= `100`, T= `101`) y hacer el cambio de árbol cuando llega la `L`, el último símbolo distinto; (2) BWT de `AMARACHACATA$`, Huffman estático con memoria nula y como Markov de orden 1, comparar longitudes promedio; (3) con $L = \dfrac{K}{1 - p^{2^K - 1}}$ hallar K_{opt} en `00000001100000000000010010000000000000001` y comprobarlo a mano; (4) canal ternario uniforme con $p(1|0) = p(2|0) = 0{,}05$: capacidad (y captura del software) y probabilidades del peor canal.

Ejercicio 1 — `CANTABALA` (Huffman dinámico). Los pasos con balanceo (Sibling Property, 6.7.1) son 3, 4, 6, 7 y 8:

Paso	Símbolo	Bits emitidos	Sibling Property	Códigos vigentes después
1	C (nuevo)	`010`	no hace falta	C= `1`, NYT= `0`
2	A (nuevo)	`0` + `000` = `0000`	no hace falta	C= `1`, A= `01`, NYT= `00`
3	N (nuevo)	`00` + `100` = `00100`	**swap** nodo(NYT) ↔ C	C= `0`, A= `11`, N= `101`, NYT= `100`
4	T (nuevo)	`100` + `101` = `100101`	**swap** nodo(NYT) ↔ C	C= `10`, A= `11`, N= `01`, T= `001`, NYT= `000`
5	A	`11`	no (A ya es el último de su peso)	A pesa 2
6	B (nuevo)	`000` + `001` = `000001`	**swap** nodo(NYT) ↔ C	C= `00`, N= `01`, A= `11`, T= `101`, B= `1001`, NYT= `1000`
7	A	`11`	**swap** A ↔ nodo de peso 2 (A pesa 3 y pasa a la rama corta)	A= `0`, C= `110`, N= `111`, T= `101`, B= `1001`
8	**L (nuevo, el último)**	`1000` + `011` = `1000011`	**swap** nodo(NYT) ↔ N: **el "cambio necesario en el árbol" que pide el enunciado**	A= `0`, N= `100`, T= `101`, C= `110`, B= `1111`, L= `11101`, NYT= `11100`
9	A	`0`	no	A pesa 4

Cadena: `010 0000 00100 100101 11 000001 11 1000011 0` = `0100000001001001011100000111110000110` $\rightarrow 3+4+5+6+2+6+2+7+1 = \mathbf{36}$ bits ✓ (coincide con la resolución de la alumna). **Comparación**: código fijo de $\lceil \log_2 6 \rceil = 3$ bits ⇒ $9 \cdot 3 = 27$ bits ⇒ compresión $1 - 36/27 = -33\%$ (**expande**: 9 símbolos, 6 de ellos distintos; el aprendizaje cuesta más de lo que ahorra). *(Los desempates del swap dependen de la convención; los 36 bits son los del resultado resuelto a mano.)*

Ejercicio 2 — BWT de `AMARACHACATA$` y comparación.

1. **BWT**: 13 rotaciones, ordenar con `$` primero, última columna ⇒ `ATHR$MCAACAAA` ✓.
2. **Huffman estático, memoria nula**: $p(A) = 6/13$, $p(C) = 2/13$, $p(\$) = p(H) = p(M) = p(R) = p(T) = 1/13$ ⇒ `A` = `1`, `C` = `010`, `$` = `011`, `H` = `0000`, `M` = `0001`, `R` = `0010`, `T` = `0011` ⇒ $L = \frac{6 \cdot 1 + 2 \cdot 3 + 3 + 4 \cdot 4}{13} = \frac{31}{13} = \mathbf{2{,}385}$ bits/símbolo (31 bits para los 13).
3. **Como Markov de orden 1** (pares sin cierre circular: 12, la cadena termina): $A \to A$ $3/5$, $A \to C$ $1/5$, $A \to T$ $1/5$; todos los demás estados tienen **un único sucesor** ($C \to A$, $T \to H$, $H \to R$, $R \to \$$, $\$ \to M$, $M \to C$ con $p = 1$).
   - Estado A: Huffman `A` = `0`, `C` = `10`, `T` = `11` ⇒ $L_A = 1{,}4$.
   - Estados con un solo sucesor: $p = 1 \Rightarrow l = \log_2 1 = 0$ bits (no hace falta enviar nada). La alumna les asignó **1 bit** (`0`) a cada uno: es válido pero **sobrante**; con esa convención: $L_M = \frac{5 \cdot 1{,}4 + 7 \cdot 1}{12} = 1{,}167$ (ella da $1{,}185$ pesando por frecuencias sobre 13, que incluye el

último `A` sin sucesor), y con 0 bits $L_M = \frac{5 \cdot 1{,}4}{12} = \mathbf{0{,}583}$.

- $H(S_M) = \frac{5}{12}\, H(\frac{3}{5}, \frac{1}{5}, \frac{1}{5}) = \frac{5}{12} \cdot 1{,}371 = 0{,}571 \leq 0{,}583$ ✓.
- **Mensaje** `ATHR$MCAACAAA`: primer símbolo con el código de memoria nula (`A` = `1`, 1 bit) + transiciones: $A \to T$ `11`, $T \to H \ldots M \to C$ (0 bits cada una, 1 bit con la convención de la alumna: total 15 bits), $C \to A$, $A \to A$ `0`, $A \to C$ `10`, $C \to A$, $A \to A$ `0`, $A \to A$ `0` ⇒ **8 bits** con la convención de 0 bits (**15** con la de 1 bit) contra **31** bits de memoria nula.

4. **Conclusión**: $L_{Huffman} = 2{,}385 > L_M = 1{,}167$ (o $0{,}583$): conocer el estado anterior baja mucho la longitud, y **la BWT ayuda porque transformó el texto en una cadena muy predecible** (casi cada símbolo tiene un único sucesor).

Ejercicio 3 — K óptimo por conteo y por fórmula. Cadena de 40 bits, 5 unos y 35 ceros; rachas de ceros antes de cada `1`: $7,\ 0,\ 12,\ 2,\ 14$ (termina en `1`: no hay racha final).

K	**Máx. por contenedor** $2^K - 1$	**Contenedores**	**Bits** $= K \cdot$**cont.**	$L = \dfrac{K}{1 - p^{2^K - 1}}$ **con** $p = \frac{35}{40} = 0{,}875$
2	3	15	**30**	$6{,}059$
3	7	9	**27**	$4{,}940$
4	**15**	**5**	**20**	$4{,}624$ **(mínimo)**
5	31	5	**25**	$5{,}081$

$K_{opt} = \mathbf{4}$: es el mínimo **por los dos caminos** (20 bits y $L = 4{,}624$), y se verifica $K + 1 = 5$ (25 bits, $5{,}081$) para poder cortar. *(p es la probabilidad de **acierto**, es decir de `0`; la fórmula usa el exponente $2^K - 1$.)* Ahorro: $1 - 20/40 = 50\%$.

Ejercicio 4 — canal ternario uniforme. $p(0|0) = 1 - 0{,}05 - 0{,}05 = 0{,}9$; matriz $\begin{bmatrix} 0{,}9 & 0{,}05 & 0{,}05 \\ 0{,}05 & 0{,}9 & 0{,}05 \\ 0{,}05 & 0{,}05 & 0{,}9 \end{bmatrix}$; $C = \log_2 3 - H(\text{fila}) = 1{,}5850 - 0{,}5690 = \mathbf{1{,}0160}$ bits (ver 2.4.1). **Peor canal**: las 9 probabilidades condicionales valen $1/3$; $H(B|A) = H(B) = \log_2 3 \Rightarrow C = 0$.

11.12 Parcial 07-10-2025, Tema 2: BWT inversa, Markov, LZW, Huffman dinámico (decodificación) y canal ternario

Enunciado: (1) `O$OOOOCLTMCRBO` es una BWT: recuperar la cadena original, entropía de Markov de orden 1 de **ambas**, comparar, ¿hay ganancia al aplicar BWT y comprimir?; comprimir la cadena transformada con **LZW** (universo ASCII); (2) descomprimir `000000100010111000110110010 0` (Huffman dinámico; tabla R= `000`, O= `001`, B= `010`, D= `011`, S= `100`); (3) canal ternario uniforme con $p(0|0) = 0{,}8$, $p(1|0) = 0{,}15$: matriz, capacidad a mano y con el software.

(1a) BWT inversa: escribir la última columna L = `O$OOOOCLTMCRBO`; ordenarla para obtener F; **pegar L a la izquierda y reordenar, $n = 14$ veces**; la fila terminada en `$` es la original ⇒ `BROTOCOMOLOCO$` ("BROTO COMO LOCO"). Comprobación: BWT directa de `BROTOCOMOLOCO$` devuelve `O$OOOOCLTMCRBO` ✓.

(1b) Entropía de Markov de orden 1 (pares sin cierre circular: la cadena termina en `$`; 13 pares; se pondera con el **número de veces que aparece cada estado como origen**):

Cadena	**Filas con ramificación**	$H(S_M) = \sum \frac{n_i}{13} H(\text{fila}_i)$
Original `BROTOCOMOLOCO$`	O (6 veces): $T, C, C, M, L, \$ \to (\frac{1}{6}, \frac{2}{6}, \frac{1}{6}, \frac{1}{6}, \frac{1}{6})$, $H = 2{,}2516$; las demás: un único sucesor ($C \to O$ siempre, etc.) ⇒ $H = 0$	$\frac{6}{13} \cdot 2{,}2516 = \mathbf{1{,}0392}$
BWT `O$OOOOCLTMCRBO`	O (5 veces): $\$, O, O, O, C \to (\frac{1}{5}, \frac{3}{5}, \frac{1}{5})$, $H = 1{,}3710$; C (2 veces): $L, R \to (\frac{1}{2}, \frac{1}{2})$, $H = 1$; las demás $H = 0$	$\frac{5}{13} \cdot 1{,}3710 + \frac{2}{13} \cdot 1 = \mathbf{0{,}6811}$

Entropía de orden 0 de ambas (misma para las dos porque la BWT es una permutación): $H(S) = 2{,}5567$ bits. **Conclusión**: la BWT baja la entropía condicional de $1{,}039$ a $0{,}681$ (-34%): agrupó iguales (`OOOO`), así que lo que sigue a cada símbolo es más predecible ⇒ **sí hay ganancia** al codificar con un compresor que use contexto (Markov, Huffman por estado, RLE, etc.).

> ⚠️ **Error de la alumna (Melani)**: planteó $H = 1 \cdot 0 + \frac{1}{2} \cdot 1 + \frac{1}{2} \cdot 1 + 1 \cdot 0 + \frac{1}{5}\log_2 5 + \frac{3}{5}\log_2 \frac{5}{3} + \frac{1}{5}\log_2 5 = 2{,}371$, es decir **sumó las entropías de las filas sin ponderar** por la frecuencia de cada estado. El resultado correcto es $0{,}681$; una entropía condicional de $2{,}371$ no tendría sentido para esa cadena (la entropía de orden 0 es $2{,}557$ y casi todos sus estados son deterministas).

(1c) LZW (diccionario inicial = 256 símbolos ASCII ⇒ códigos de 9 bits cuando el diccionario supera 256):

- `O$OOOOCLTMCRBO` ⇒ `O` (79) `$` (36) `O` (79) `OO` (258) `O` (79) `C` (67) `L` (76) `T` (84) `M` (77) `C` (67) `R` (82) `B` (66) `O` (79) = **13 códigos**; entradas nuevas: 256= `O$`, 257= `$O`, 258= `OO`, 259= `OOO`, 260= `OC`, 261= `CL`, 262= `LT`, 263= `TM`, 264= `MC`, 265= `CR`, 266= `RB`, 267= `BO`.
- Con 9 bits por código: $13 \cdot 9 = 117$ bits contra $14 \cdot 8 = 112$ bits en ASCII ⇒ **no comprime** (cadena demasiado corta: casi no hay repeticiones). La original da también 13 códigos (`OC` se repite una vez). **Lectura**: la ganancia de la BWT se ve con textos largos; en una cadena de 14 símbolos con LZW no alcanza.

(2) Huffman dinámico, decodificación de `0000001000101110001101100100` (fijo: R= `000` , O= `001` , B= `010` , D= `011` , S= `100` ; 28 bits):

Paso	Bits leídos	Interpretación	Símbolo	Sibling Property
1	`000`	árbol vacío ⇒ fijo	**R**	no hace falta
2	`0` + `001`	NYT= `0` + fijo	**O**	no hace falta
3	`00` + `010`	NYT= `00` + fijo	**B**	**swap** nodo(NYT) ↔ R (R pasa a `0`)
4	`11`	O ya está (`11`)	**O**	**swap** O ↔ R (O sube a `0` , R pasa a `11`)
5	`100` + `011`	NYT= `100` + fijo	**D**	**swap** nodo(NYT) ↔ R
6	`0`	O= `0`	**O**	no (O ya es el líder de su peso)
7	`1100` + `100`	NYT= `1100` + fijo; **último símbolo distinto de la fuente**	**S**	**swap** nodo(NYT) ↔ R (el "cambio necesario" del enunciado)

Bits: $3+4+5+2+6+1+7=28$ ✓. **Texto:** `ROBODOS` **("ROBO DOS")**. Comprobación: comprimir `ROBODOS` con las mismas reglas devuelve exactamente la cadena dada. Árbol final (pesos): O_3 (código `0`) y nodo $4=[\,[S_1,D_1]_2,\,[R_1,B_1]_2\,]$.

(3) Canal ternario uniforme, $p(0|0)=0{,}8$**,** $p(1|0)=0{,}15$**:** $p(2|0)=0{,}05$; **uniforme** ⇒ las otras filas son permutaciones de la primera: $\begin{bmatrix}0{,}8 & 0{,}15 & 0{,}05\\ 0{,}05 & 0{,}8 & 0{,}15\\ 0{,}15 & 0{,}05 & 0{,}8\end{bmatrix}$. $C=\log_2 3-H(0{,}8;0{,}15;0{,}05)=1{,}5850-0{,}8842=\mathbf{0{,}7008}$ bits, con entrada equiprobable (cuenta completa en 2.4.1).

> ⚠️ **Dos errores vistos**: (a) **Melani** puso $1/3$ en las filas 2 y 3 (eso **no** es un canal uniforme) y llegó a $I=0{,}158$, que ni siquiera es la capacidad de esa matriz; (b) **Pedro**: la captura del software dice $C=0{,}6986$ con distribución óptima $(0{,}318;0{,}333;0{,}349)$ y I uniforme $=0{,}6960$, lo que delata que cargó una matriz **no** uniforme (filas $[0{,}8;0{,}15;0{,}05]$, $[0{,}15;0{,}8;0{,}05]$, $[0{,}05;0{,}15;0{,}8]$: columnas desparejas). En un canal uniforme el óptimo es siempre equiprobable. **Chequeo**: si el software no te devuelve $p(x)$ uniforme para un canal uniforme, la matriz está mal cargada.

11.13 PPM-C: trayectorias cortas (Temas 3 y 4 de 2025)

Convenciones del apunte (6.8): una tabla vacía **no cuesta nada** (ESC con $p=1$); $P(ESC)=\dfrac{U}{total+U}$ con U = símbolos distintos del contexto; el orden -1 da $1/64$.

Tema 3 — `ABRA` **, PPM-C de orden 2.** a) **Contextos creados**: orden 2: `AB` , `BR` ; orden 1: `A` , `B` , `R` ; orden 0: la tabla global. b) **Eventos**:

Símbolo	Eventos	Probabilidades (cadena de ESC)
A	ESC ESC ESC `A` (1/64)	$1\cdot 1\cdot 1\cdot \frac{1}{64}$
B	ESC ESC ESC `B` (1/64)	$1\cdot 1\cdot \frac{1}{2}\cdot \frac{1}{64}$ (orden 0 conoce solo A: $P(ESC)=\frac{1}{1+1}$)
R	ESC ESC ESC `R` (1/64)	$1\cdot 1\cdot \frac{2}{4}\cdot \frac{1}{64}$ (orden 0 conoce A y B)
A	ESC ESC `A` (1/6)	$1\cdot 1\cdot \frac{1}{6}$ (orden 0 conoce A, B, R una vez cada una: $total=3$, $U=3\Rightarrow P(A)=\frac{1}{3+3}$)

c) **El último `A` se codifica en el contexto de orden 0** (`BR` y `R` nunca vieron un sucesor), con $P=\mathbf{1/6}$ ✓ (las alumnas lo escribieron igual).

Tema 4 — `ABA` **, PPM-C de orden 1.** a) **Contextos**: orden 1: `A` (se crea al leer `B`) y `B` (se crea al leer la 2.ª `A`); orden 0. b) **Eventos**: A: ESC ESC `A` (1/64); B: ESC ESC `B` (1/64) (con $P(ESC_0)=\frac{1}{2}$); A: ESC `A` (1/4) (el contexto `B` está vacío, el orden 0 conoce A y B una vez cada uno: $total=2$, $U=2\Rightarrow P(A)=\frac{1}{2+2}$). c) **El último `A` se codifica en el contexto de orden 0 con probabilidad $1/4$** (justificación: el contexto de orden 1 es `B` , que todavía no tiene sucesores).

(Pregunta intercalada de la hoja de Tema 3, "alfabeto de 32 símbolos equiprobables": un intervalo final de $P=1/32$ requiere $\lceil\log_2 32\rceil=5$ bits; es lo que cuesta cada símbolo en el orden -1 de ese alfabeto.)

11.14 Teóricas de 2025 (nuevas) y sus trampas

1. **Fuente de $q=6$ (cara de un dado) con $L_{Huffman}=L_{Shannon}$, base 2, y Kraft.** Distribución: $\frac{1}{2},\frac{1}{4},\frac{1}{8},\frac{1}{16},\frac{1}{32},\frac{1}{32}$ (potencias de 2). Longitudes $1,2,3,4,5,5$ en **Huffman y en Shannon** ⇒ $L=H=1{,}9375$ bits. **Kraft**: $\frac{1}{2}+\frac{1}{4}+\frac{1}{8}+\frac{1}{16}+\frac{1}{32}+\frac{1}{32}=\mathbf{1}$. ⚠️ Dos errores de las resoluciones: (a) **la distribución uniforme NO sirve**: con $p=\frac{1}{6}$, Huffman da $L=\frac{16}{6}=2{,}667$ (longitudes $2,2,3,3,3,3$) y Shannon da 3 (todas $\lceil\log_2 6\rceil=3$); además con 7 símbolos uniformes: Huffman $2{,}857$ y Shannon 3. Solo coinciden cuando $p_i=2^{-l_i}$; (b) con longitudes todas 3 la suma de Kraft es $6/8=0{,}75\le 1$ ($\le$, no "$\ge$"), pero ese código no es compacto. *(Para $q=7$ también sirve $\frac{1}{2},\frac{1}{4},\frac{1}{16},\frac{1}{16},\frac{1}{16},\frac{1}{32},\frac{1}{32}$: longitudes $1,2,4,4,4,5,5$ en ambos, $L=H=2{,}0625$, Kraft $=1$.)*
2. **LZ/LZW y Huffman: ¿cuál va primero?** Primero el de **diccionario** (LZ/LZW: saca la redundancia por repetición de subsecuencias), después el **estadístico** (Huffman/Shannon: asigna códigos cortos a lo frecuente sobre la secuencia ya "aplanada"). *(Las dos hojas de Tema 4 y Tema 3 lo respondieron bien.)*

3. **Alta redundancia o alta entropía para comprimir más** ⇒ **alta redundancia** (baja entropía respecto de H_{max}): hay más estructura que eliminar.
4. **Objetivo de la BWT** (Tema 3 autocorrección): agrupar símbolos iguales para facilitar la compresión posterior (RLE, Huffman, Markov por estado, LZ).
5. **Canal binario** $\begin{bmatrix} 0{,}95 & 0{,}05 \\ 0{,}3 & 0{,}7 \end{bmatrix}$: **no es simétrico** ($p(1|0) = 0{,}05 \neq p(0|1) = 0{,}3$); conviene transmitir con mayor probabilidad el **0** (su fila tiene menos ruido: $H(0{,}05) = 0{,}286$ vs. $H(0{,}3) = 0{,}881$). Para la capacidad no se puede usar $1 - H(\bar{p})$ con $\bar{p} = 0{,}175$ (da $0{,}331$ y es incorrecto): hay que maximizar I ⇒ $C = \mathbf{0{,}3731}$ con $p(0) \approx 0{,}544$ (con $p(0) = 0{,}5$ da $0{,}3706$).

11.15 Discretización con error "menor a x" (PCM de 16 bits y ADPCM)

Enunciado tipo (Tema 2 autocorrección): señal de -5 V a $+5$ V en **PCM con 16 bits**; error de cuantificación al aproximar con el **valor medio** de cada segmento; con **DPCM/ADPCM** el rango baja a $\pm 0{,}5$ V: ¿cuántos bits para mantener el mismo error?

- **PCM**: $\Delta = 10/2^{16} = 1{,}526 \cdot 10^{-4}$ V ⇒ error $= \pm\Delta/2 = \pm\mathbf{7{,}63 \cdot 10^{-5}}$ V (la hoja lo dio como $\pm 0{,}000075$ ✓).
- **ADPCM** (rango 1 V, mismo Δ): niveles $= 1/1{,}526 \cdot 10^{-4} = 6553{,}6 \to \lceil \log_2 6553{,}6 \rceil = \mathbf{13}$ bits ($2^{12} = 4096 < 6553{,}6 \leq 8192 = 2^{13}$) ✓.

Variante "encontrar el error menor a $0{,}0001$ V" (Tema 1 autocorrección): ancho del segmento $= 2e = 0{,}0002$ V ⇒ niveles $= 10/0{,}0002 = 50\,000$ ⇒ $2^{15} = 32768 < 50\,000 \leq 65\,536$ ⇒ **16 bits**; con ADPCM de $\pm 0{,}5$ V: $1/0{,}0002 = 5000$ ⇒ $2^{12} = 4096 < 5000 \leq 8192$ ⇒ **13 bits**. (Misma lógica que 3.5.)

> ⚠️ **Error visto (Tema 3, hoja de «matil»)**: en la parte B dividió el rango 1 V **por el error máximo** $0{,}00488$ y obtuvo 205 niveles ⇒ 8 bits. Hay que dividir por el **ancho** $= 2\cdot$ error $= 0{,}00977$: $1/0{,}00977 = 102{,}4$ ⇒ **7** bits (con 8 bits el error sería menor al necesario). Es el mismo error del "400 vs. 200 niveles" de 3.5.

11.16 Variantes de las planillas "Autocorrección" 2025 (Temas 1 a 3): qué se verificó y qué dudar

- **Tema 3, fuente** `a e i i o o u u a e i i` (12 símbolos): $p = (2, 2, 4, 2, 2)/12$ ⇒ $H = 4 \cdot \frac{1}{6}\log_2 6 + \frac{1}{3}\log_2 3 = \mathbf{2{,}2516}$ (la hoja dice $2{,}246$ por redondeo) ⇒ $R = 1 - H/H_{max} = 3{,}03\%$. **Huffman dinámico** de esos 12 símbolos con el código fijo i= `000` , a= `001` , e= `010` , o= `011` , u= `100` : con la regla de swap del apunte (FGK, 6.7.1) salen **48 bits** contra 36 del código fijo (la hoja da 45; la diferencia es de convención de desempate). En ambos casos **expande**: es normal con 12 símbolos y 5 distintos.
- **Tema 1, fuente de 30 símbolos** con $p = (7, 6, 6, 6, 5)/30$: $H = 2{,}3139$, $R = 0{,}35\%$ (la hoja escribe la **redundancia absoluta** $H_{max} - H$ en bits: $0{,}008$; recordá aclarar cuál de las dos definiciones usás, ver 10.1). Huffman independiente: longitudes $2, 2, 2, 3, 3$ ⇒ $L = 71/30 = 2{,}367$ ✓ ⇒ $1 - 71/90 = 21{,}1\%$ ✓. Su matriz de Markov no tiene ceros ⇒ **ergódica** (criterio práctico de la cátedra: ninguna entrada nula ⇒ regular). ⚠️ La hoja da $H(S_M) = 2{,}41 > H(S) = 2{,}31$ y una **redundancia negativa**: **es imposible** ($H(S_M) \leq H(S)$ siempre). El error está en la fila de $e/o/u$: con probabilidades $(\frac{1}{6}, \frac{1}{6}, \frac{1}{6}, \frac{2}{6}, \frac{1}{6})$ la entropía es $4 \cdot \frac{1}{6}\log_2 6 + \frac{2}{6}\log_2 3 = 2{,}2516$ (la hoja puso $4 \cdot \frac{2}{6}\log_2 3$); con ese arreglo $H(S_M) \approx 2{,}23$.
- **Tema 2, canal** $0{,}8/0{,}1/0{,}1$ **y Tema 3, canal** $0{,}7/0{,}2/0{,}1$: resultados de la hoja ($1{,}278$ y $1{,}2336$) **incorrectos**; correctos $0{,}6630$ y $0{,}4282$ (ver 2.4.1).
- **LZ con ventana de 8 caracteres (Tema 2, 12 símbolos)**: la hoja cuenta $8 + 9 + 8 + 9 + 9 + 9 + 9 + 9 + 8 = 78$ bits con campos *flag*(1)+desplazamiento(4)+longitud(3) y $64 + 78 = 142$ vs. 160 bits sin comprimir ($11{,}25\%$), con la nota "me informaron que esto es incorrecto". **No se re-resolvió acá**: el costo por unidad depende de cuántos bits da el enunciado para desplazamiento y longitud (ver 6.9 y 10.6).
- **Redundancia con otra definición**: varias hojas usan $R = H_{max} - H$ (en bits) en vez de $1 - H/H_{max}$ (en %). Decí cuál usás y, si podés, dá las dos.

*(Material de 2025 que ya estaba en el apunte y se descartó por repetido: el enunciado del parcial **2022 Tema I** en foto de WhatsApp (`amnr$aaag`, Shannon/Huffman, BSC 0,85), el parcial **Nanni Rebollo** / `VISTO-KLE.pdf` (copia idéntica del que ya figuraba) y el Tema 1 de Moya (11.10).)*

---

12. Errores que el docente marcó (y cómo evitarlos)

1. **Shannon**: si $\log_2(1/p)$ es entero se usa ese entero, **no** se suma 1. Siempre $l_i = \lceil \log_2(1/p_i) \rceil$.
2. **Discretización**: con valor medio el error es la mitad del ancho ⇒ ancho $= 2\cdot$ error (200 segmentos y 8 bits en la versión A, no 400 y 9). En DPCM se usa el rango de las **diferencias**.
3. **Predictiva**: verificar $K + 1$ (con $K = 4$ ya empeora); no agregar un `1` al principio de la cadena.
4. **Canal**: la capacidad se calcula con entrada equiprobable (BSC); no mezclar $p(0) = 0{,}7$ si solo piden C. Peor canal ternario ⇒ $C = 0$, **no** $1{,}0566$. $I(A;B)$ **nunca** es negativa (una resolución dio $-0{,}115$). I nunca supera $\log_2 q$ (otra dio $8{,}7178$ para un canal binario).
5. **Aritmético**: si con la precisión pedida cota inferior = cota superior, **no se puede seguir** ("no se puede discernir un valor entre ellas").
6. **Markov**: dividir cada fila por las apariciones del estado **origen** (no por n); armar bien la secuencia circular; contestar la ergodicidad con el criterio de accesibilidad (5.2).
7. **Fano ≠ Huffman**: leer cuál algoritmo piden (en el recuperatorio hicieron Huffman donde pedían Fano).

8. **Contestar todas las preguntas del enunciado** (en el parcial de Arce se perdieron puntos por no responder las preguntas de cada inciso; la nota fue 7/10).
9. **Justificar las teóricas** ("¿Por qué?"): decir "sería mucho menor" no alcanza; hay que explicar por qué BWT mejora la compresión de Markov (sección 9 #12).
10. **Peor canal vs. mejor canal**: leer cuál piden. Pidieron el mejor ternario (unos en la diagonal) y el alumno hizo el peor.
11. **Distribución que iguala Huffman y Shannon**: debe **sumar 1** y ser potencias de 2 (1/2, 1/4, 1/8, 1/16, 1/16…).
12. **Huffman dinámico**: no olvidar las rondas de "balanceo" (Sibling Property) de los pasos 3, 5, 7 y 9 de `ABRACADABRA` (y 3, 4 y 5 en el ejemplo `aeieo` del Tema 2); explicar **en qué pasos** se aplica **y** su objetivo (el árbol sigue siendo de Huffman), son dos respuestas (6.7.1); y usar el resultado resuelto a mano (**35 bits**) como referencia.
13. **Entropía de Markov (y ruido $H(B|A)$) sin ponderar**: se promedia con la frecuencia/probabilidad de cada estado (o de cada entrada), no se suman las filas (11.12, 2.4.1).
14. **Canal ternario**: $H(B|A)$ se pesa por las **tres** filas ($3 \cdot \frac{1}{3}$); la capacidad de un canal uniforme es $\log_2 q - H(\text{fila})$ (resultados $0{,}7008$, $0{,}6630$, $0{,}4282$, $1{,}016$); "uniforme" = filas permutadas, no $1/3$ en las otras (2.4.1, 11.12).
15. **Huffman por bigramas**: unir de **a dos**, y comprobar que el código es instantáneo y que Kraft $= 1$ (11.10).
16. **Teórica "Huffman = Shannon"**: la distribución **uniforme no sirve** salvo que q sea potencia de 2; usar $p_i = 2^{-l_i}$ (11.14).
17. K **óptimo**: contar bits **y** verificar con $L = K/(1 - p^{2^K - 1})$, con p = prob. de ceros (11.11).
18. **Discretización con valor medio**: dividir por el **ancho** $= 2\cdot$error, no por el error (11.15).

Nota de cobertura

Los parciales corregidos de 2023 usados como referencia para este apunte fueron resoluciones manuscritas de estudiantes de la misma batería que el examen modelo (4 variantes), más un Recuperatorio Tema 2 con un BSC de $p = 1/8$ y decodificación de Huffman dinámico. Algunas de esas resoluciones contienen errores aritméticos (el caso más claro: una $I(A;B)$ negativa, matemáticamente imposible). En este apunte **todos los resultados numéricos fueron recalculados de forma independiente** —los de la sección 6, además, verificados con cálculo exacto por computadora (fracciones para Huffman/Shannon/Fano/aritmética, y una simulación completa con decodificación verificada para el Huffman dinámico)— por lo que no arrastran esos errores.